PARADIGMA ORIENTADO A OBJETOS · UADE · 2C 2026

Sprints de Alquimia

Qué hace cada una y cuándo, para el TP del depósito de químicos.

Hoy

Próxima pre-entrega

Lunes 19/10 · partida por consola (sin nota)

28 días

Entrega final y defensa

Lunes 16/11 (Grupo 1) · 30/11 si son Grupo 2

Sprint 4

La ventana

Del 19/10 al 25/10

1 · EL PLAN

Calendario

Un tubo por semana. Abajo de cada uno, lo que se entrega o se tiene que cumplir para pasar al siguiente.



HECHO

Sprint 1 · 28/9 al 4/10

Análisis y UML

Diagrama de clases, secuencias y boceto

El profe ve el UML el 5/10

HECHO

Sprint 2 · 5/10 al 11/10

El modelo en código

Químicos, cola y depósito, sin ventana

Las reglas andan por consola · el lunes 12/10 es feriado

HECHO

Sprint 3 · 12/10 al 18/10

El juego por consola

Pedidos, turnos, retiros y práctica de Swing

Pre-entrega sin nota · lunes 19/10

ESTAMOS ACÁ

Sprint 4 · 19/10 al 25/10

La ventana

Swing conectado al modelo y ArrayList

Pre-entrega sin nota · lunes 26/10

Sprint 5 · 26/10 al 1/11

Que se entienda

Puntaje, top 5, balanceo y excepciones

Pre-entrega sin nota · lunes 2/11

Sprint 6 · 2/11 al 8/11

UML, archivos e hilo

Archivos, paquetes, hilo, UML y memoria

Lunes 9/11 feriado · sin pre-entrega

Sp

Me

1 · EL PLAN

Tareas por sprint

Cada tarea tiene «Cómo hacerlo» con los pasos. Los tildes quedan guardados solo en este navegador.

00 **Eq** Todas | 01 **S_1** Sofia1 | 02 **S_2** Sofia2 | 03 **Ly** Lyssie | 04 **Lz** Luz

Tocá tu ficha para ver solo tus tareas y las del equipo.

SPRINT 1 · 28/9 AL 4/10

0 de 10 hechas — Terminado

Análisis y UML ▲

El diseño queda dibujado antes de escribir una línea de código, como pide el profe. El lunes 5/10 se le muestra el UML.

Primera reunión · las cuatro juntas, 1 hora, por Meet o presencial

- [ ] **Todas**

Leer juntas el CLAUDE.md completo, sobre todo las secciones 2, 3 y 4.

− Cómo hacerlo

PASOS

1. Una comparte pantalla y lee en voz alta; las demás cortan cuando algo no se entiende.
2. Sección 2: repasar con un ejemplo la regla de compatibilidad. Solo se compara contra el último de la cola. Permitido: {Inflamable, Oxidante} al lado de {Acido, Base}. Prohibido: dos iguales, Inflamable con Oxidante, Acido con Base.
3. Sección 2: repasar la cola. El nuevo entra siempre al final; si se saca uno del medio, los de atrás se corren y pueden quedar pegados dos incompatibles. Es a propósito.
4. Sección 3: las doce clases y qué hace cada una.
5. Sección 4: las cuatro reglas de diseño. La más importante: nada de `if` preguntando el tipo de químico fuera de las clases de químico, y la ventana nunca decide reglas.

ESTÁ LISTA CUANDO

- Cada una puede contar, sin mirar, qué pasa si se pone una base detrás de un ácido y qué pasa al sacar un químico del medio.

- [ ] **Todas**

Instalar todas lo mismo: el JDK, IntelliJ IDEA, GitHub Desktop y draw.io. Crear el repositorio y que cada una suba un commit de prueba.

− Cómo hacerlo

PASOS

1. **Versión de Java: Eclipse Temurin 25.** El profe no pidió ninguna. Java 25 es la versión LTS (de soporte largo) más nueva, Temurin es gratis y recibe actualizaciones, y todo lo que se ve en la cursada funciona igual. Anotarlo en la sección 5 del CLAUDE.md.
2. **Instalar IntelliJ IDEA** desde jetbrains.com/idea (la edición gratuita alcanza).
3. **Instalar el JDK desde IntelliJ**, así las cuatro tienen exactamente el mismo: File › Project Structure › Project › SDK › Download JDK. En la ventana que se abre, elegir **Version: 25** y **Vendor: Eclipse Temurin**, y tocar Download. Las cuatro eligen exactamente eso.

4. **Instalar GitHub Desktop** desde desktop.github.com e iniciar sesión con la cuenta de GitHub. Quien no tenga cuenta, la crea en github.com.
5. **draw.io** no se instala: se usa desde app.diagrams.net.
6. **Crear el repositorio** (lo hace una sola) y **clonarlo** (las otras tres): ver la guía de Git, pasos 1 a 3.
7. **Commit de prueba:** cada una edita el README con su nombre, hace commit y push (guía de Git).

TIENE QUE QUEDAR

- Un repositorio en GitHub con las cuatro como colaboradoras y un proyecto de IntelliJ adentro, con la carpeta `src`.

ESTÁ LISTA CUANDO

- En github.com, en la pestaña Commits del repositorio, aparecen cuatro commits, uno de cada una.
- Las cuatro abren el proyecto en IntelliJ y ven Eclipse Temurin 25 en File › Project Structure › Project › SDK.

> Si a alguna le aparece otra versión de Java, cambiarla antes de seguir: con versiones distintas aparecen errores que parecen del código y no lo son.

- [ ] **Todas**

Decidir cómo sabe cada químico con quién es compatible. Antes de elegir, releer la pregunta de la defensa sobre un quinto tipo de químico (imaginario: el juego tiene 4).

− **Cómo hacerlo**

PASOS

1. **Ojo, esto confunde:** el juego tiene 4 tipos (`Acido`, `Base`, `Inflamable`, `Oxidante`) y no se agregan más. El «quinto tipo» **no existe en el juego**: es un caso imaginario que usa el CLAUDE.md (secciones 4.1 y 10) para probar si el diseño está bien hecho. La pregunta es: si mañana el profe dice «agreguen un tipo nuevo», ¿cuántas clases que ya existen hay que modificar? Lo ideal es ninguna: solo se crea la clase nueva.
2. Releer la sección 4.1: la compatibilidad se resuelve con `esCompatibleCon(Quimico otro)`, abstracto en `Quimico`, y el criterio es que **agregar un quinto tipo de químico no obligue a modificar ninguna clase existente**.
3. Tener en cuenta cómo se usa: en `puedeGuardar()` se llama `ultimo.esCompatibleCon(nuevo)`. Es decir, el que decide es el químico **viejo**, que ya estaba en la cola.
4. Probar cada idea con la pregunta de la defensa: si mañana el profe pide agregar un quinto tipo, ¿el químico viejo sabe responder sin que se toque su clase? Si la respuesta es "hay que agregarle un caso a cada clase", esa idea no cumple el criterio.
5. Anotar la decisión y el porqué en el CLAUDE.md, en la sección 4.1: es exactamente lo que van a preguntar en la defensa.

TIENE QUE QUEDAR

- Una frase en el CLAUDE.md que diga cómo decide cada químico y por qué un quinto tipo no obliga a tocar las clases que ya existen.

ESTÁ LISTA CUANDO

- Las cuatro pueden contestar la pregunta del quinto tipo con la decisión tomada.

Todas

Crear en Google Drive un archivo de draw.io para el diagrama de clases, con una página por parte.

Cómo hacerlo

PASOS

1. Seguir la guía de draw.io, pasos 1 y 2.
2. Crear cuatro páginas: "Químicos", "Contenedores", "Juego y vista" y "Secuencias". Al final se agrega "Diagrama unido".

TIENE QUE QUEDAR

- Un solo archivo en Drive, compartido como editoras con las cuatro.

ESTÁ LISTA CUANDO

- Las cuatro lo abren y pueden mover una figura.

Tareas

Sofia1

Diagrama de clases, parte químicos: `Quimico` abstracta y sus cuatro subclases, con atributos, métodos, visibilidad (`+` , `-` , `#`) y la flecha de herencia. Marcar `esCompatibleCon()` como abstracto.

Con: draw.io

Cómo hacerlo

PASOS

1. Trabajar en la página "Químicos" (guía de draw.io).
2. Dibujar `Quimico` con el nombre en cursiva (así se marca una clase abstracta en UML).
3. Atributos: lo que la sección 3 dice que sabe un químico: si necesita frío. Por ejemplo `# necesitaFrio: boolean` (con `#` porque es `protected` y lo usan las subclases).
4. Métodos: `+ esCompatibleCon(otro: Quimico): boolean` en cursiva (abstracto) y `+ necesitaFrio(): boolean`. Agregar lo que haya salido de la decisión de compatibilidad de la primera reunión.
5. Dibujar `Acido` , `Base` , `Inflamable` y `Oxidante` , cada una con `+ esCompatibleCon(otro: Quimico): boolean` sin cursiva (lo implementan).
6. Unir cada subclase con `Quimico` con la flecha de herencia: línea continua con triángulo vacío apuntando a `Quimico` .

TIENE QUE QUEDAR

- Cinco clases, cuatro flechas de herencia, visibilidad en cada atributo y método.

ESTÁ LISTA CUANDO

- Otra del grupo, mirando solo el dibujo, puede decir qué método tiene que escribir cada subclase.

Lyssie

Diagrama de clases, parte contenedores: `Contenedor` abstracta, `Heladera` , `Estante` y `Deposito` . Dibujar las relaciones con su multiplicidad: un depósito tiene seis contenedores y

cada contenedor, hasta cuatro químicos.

Con: draw.io

Cómo hacerlo

PASOS

1. Trabajar en la página "Contenedores" (guía de draw.io).
2. `Contenedor`, en cursiva. Atributos de la sección 5: `# quimicos: Quimico[4]` y `# cantidad: int`; y el código del contenedor (`H1` … `E3`), que usa el depósito para encontrarlo.
3. Métodos de `Contenedor` (secciones 3 y 4.3): `+ estaLleno(): boolean`, `+ puedeGuardar(nuevo: Quimico): boolean`, `+ guardar(nuevo: Quimico): boolean`, `+ sacar(posicion: int): Quimico`, y uno abstracto para la temperatura, como `aceptaTemperatura(q: Quimico): boolean` (lo usa `puedeGuardar()` en el CLAUDE.md).
4. `Heladera` y `Estante` heredan de `Contenedor` y solo implementan el método de temperatura.
5. `Deposito`: atributo `- contenedores: Contenedor[6]` y el último motivo de error; métodos para encontrar un contenedor por código, guardar, retirar y `+ getUltimoMotivo(): String`.
6. Relaciones: de `Deposito` a `Contenedor` con multiplicidad 1 y 6; de `Contenedor` a `Quimico` con 1 y 0..4. Escribir los números en cada punta de la línea.
7. El tipo de rombo lo decide el grupo y lo tiene que poder justificar. Una lectura posible: composición (rombo lleno) entre depósito y contenedores, porque el depósito los crea y viven con él; agregación (rombo vacío) entre contenedor y químicos, porque los químicos entran y salen.

TIENE QUE QUEDAR

- Cuatro clases con herencia de `Heladera` y `Estante`, y dos relaciones con multiplicidad escrita en las dos puntas.

ESTÁ LISTA CUANDO

- Mirando el dibujo se lee "un depósito tiene 6 contenedores y un contenedor tiene de 0 a 4 químicos".

Luz

Diagrama de clases, parte juego y vista: `Juego`, `VentanaJuego` y `PanelContenedor`, y a quién conoce cada una. Un boceto de la pantalla: seis contenedores, el pedido arriba y dónde aparece el mensaje.

Con: draw.io

Cómo hacerlo

PASOS

1. Trabajar en la página "Juego y vista" (guía de draw.io).
2. `Juego` (sección 3): genera el próximo pedido, lleva el turno y el puntaje y detecta la derrota. Atributos: el depósito, el turno, el puntaje y el pedido actual (si es dejar o retirar, y qué químico o qué tipo y temperatura). Métodos para generar el pedido, pasar al siguiente turno y saber si terminó.
3. `VentanaJuego`: conoce al `Juego` y al `Deposito` para preguntarles, y tiene 6 `PanelContenedor`. Métodos: `actualizarPantalla()` y `mostrarMensaje(texto)`.
4. `PanelContenedor`: conoce su `Contenedor` para dibujarlo con sus 4 casillas.

5. Dibujar las flechas de asociación: `Juego` → `Deposito`; `VentanaJuego` → `Juego` y → `Deposito`; `VentanaJuego` → `PanelContenedor` (1 a 6); `PanelContenedor` → `Contenedor`.
6. Boceto de la pantalla en otra página ("Boceto"): con rectángulos alcanza. Arriba, el pedido actual (qué químico y si necesita frío o calor). En el medio, tres heladeras y tres estantes, con cuatro casillas cada uno y su nombre. Abajo, el lugar del mensaje de derrota, el turno y el puntaje.

TIENE QUE QUEDAR

- Tres clases con sus flechas y un boceto con los seis contenedores, el pedido y el mensaje.

ESTÁ LISTA CUANDO

- Se ve que `VentanaJuego` no tiene ningún método que decida una regla: solo pregunta y muestra.

☐ **Sofia2**

Dos diagramas de secuencia: guardar un químico y retirar uno, desde el clic hasta la respuesta. Tienen que mostrar que la ventana pregunta y el modelo decide.

Con: draw.io

− Cómo hacerlo

PASOS

1. Trabajar en la página "Secuencias" (guía de draw.io). Cada objeto es una línea de vida vertical; cada llamada, una flecha horizontal con el nombre del método; cada respuesta, una flecha punteada de vuelta.
2. **Guardar**, tomando el orden del CLAUDE.md (4.2 y 4.3): la jugadora hace clic en un contenedor → `VentanaJuego` llama `deposito.guardar(quimicoActual, "H1")` → `Deposito` encuentra el contenedor H1 → llama `puedeGuardar(nuevo)` → el contenedor revisa si está lleno, si acepta la temperatura y llama `ultimo.esCompatibleCon(nuevo)` → vuelve `true` o `false` hasta la ventana → si es `true`, `juego.siguienteTurno()`; si es `false`, `mostrarMensaje(deposito.getUltimoMotivo())` → `actualizarPantalla()`.
3. **Retirar**: clic en una casilla → la ventana le pide al depósito retirar de ese contenedor y esa posición → el depósito revisa que sea del tipo y la temperatura pedidos → el contenedor saca el químico y corre los de atrás → se revisa si los dos que quedaron pegados son compatibles → vuelve el resultado → la ventana muestra y actualiza.
4. Usar los mismos nombres de métodos que el diagrama de clases. Si hace falta uno que no está, avisarle a la dueña de esa parte.

TIENE QUE QUEDAR

- Dos diagramas donde ninguna flecha de decisión sale de la ventana.

ESTÁ LISTA CUANDO

- Siguiendo las flechas se puede contar, sin el código, qué pasa cuando alguien pone una base detrás de un ácido.

☐ **Sofia1**

Sábado 3/10: juntar las tres partes en un solo diagrama de clases y revisar que los nombres coincidan.

Con: draw.io

− Cómo hacerlo

PASOS

1. Crear la página "Diagrama unido" y copiar las tres partes.
2. Revisar que cada clase aparezca una sola vez (por ejemplo, `Quimico` aparece en químicos y en contenedores).
3. Revisar que los nombres de métodos sean los mismos que usan las secuencias de Sofia2.
4. Contar las clases: tienen que ser doce (las diez del modelo más `VentanaJuego` y `PanelContenedor`).
5. Avisar por WhatsApp lo que no coincide, para que cada dueña lo corrija antes del domingo.

TIENE QUE QUEDAR

- Un diagrama con las doce clases y todas sus relaciones.

ESTÁ LISTA CUANDO

- Los nombres del diagrama unido y de las secuencias son iguales, letra por letra.

☐ **Todas**

Domingo 4/10: cada una explica una parte del diagrama que no dibujó.

Con: Meet con pantalla compartida

− **Cómo hacerlo**

PASOS

1. Una comparte pantalla con el diagrama unido.
2. Cada una explica una parte ajena, por ejemplo: Luz explica químicos, Sofia1 contenedores, Lyssie juego y vista, Sofia2 las relaciones del depósito.
3. La dueña de la parte corrige solo si la explicación es incorrecta.
4. Anotar lo que quedó confuso y corregirlo esa noche.
5. Exportar el diagrama unido a PDF o PNG para llevarle al profe el lunes (guía de draw.io).

ESTÁ LISTA CUANDO

- Las cuatro explicaron una parte ajena sin ayuda.

CRITERIO PARA PASAR AL SPRINT 2

El diagrama de clases tiene las doce clases con sus relaciones, están las dos secuencias y las cuatro pueden explicar cualquier parte.

SPRINT 2 · 5/10 AL 11/10

0 de 10 hechas **Terminado**

El modelo en código ▲

Se programa lo que dice el diagrama y las reglas funcionan por consola, sin ventana. El diagrama es el mapa: si el código necesita algo distinto, primero se corrige el diagrama.

☐ **Todas**

Lunes 5/10: mostrarle el UML al profe y anotar lo que pida cambiar. Sofia1 pasa al diagrama esos cambios y los del boceto antes del martes.

Con: draw.io

Cómo hacerlo

PASOS

1. Llevar el diagrama unido y las dos secuencias (en la computadora o impresos).
2. Una anota textual lo que dice el profe, en una lista numerada en el grupo de WhatsApp.
3. Sofia1 pasa cada cambio al diagrama antes del martes y tacha el número de la lista.
4. Sofia1 suma también lo del boceto (sección 3 del CLAUDE.md del boceto): `Visitante`, `TablaPuntajes`, `getImagen()`, `vaciar()`, `canjear()`, los pendientes en `Juego` y las 5 clases de vista (`VentanaJuego`, `PanelMostrador`, `PanelDeposito`, `PanelFinal` y `PanelContenedor`).
5. Contar las clases: tienen que ser 17 (12 del modelo y 5 de la vista), no 12.
6. Si el profe pide cambios grandes, aplicar el primer riesgo de la tabla del final: se corrige lunes y martes y el código arranca el miércoles.

TIENE QUE QUEDAR

- El diagrama corregido y una lista de cambios tachada.

ESTÁ LISTA CUANDO

- No queda ningún número de la lista sin tachar.

Sofia1

`Quimico` abstracta, `Acido` y `Base`, con `getImagen()`. Subir el esqueleto de `Quimico` el primer día, para que las demás puedan compilar.

Con: IntelliJ IDEA y GitHub Desktop

Cómo hacerlo

PASOS

1. Crear la rama `quimico-acido-base` (guía de Git).
2. **Primer día:** crear `Quimico.java` (guía de IntelliJ): `public abstract class Quimico`, con el atributo de temperatura (`protected boolean necesitaFrio;`), un constructor que lo reciba (`this.necesitaFrio = necesitaFrio;`), `public boolean necesitaFrio()` y `public abstract boolean esCompatibleCon(Quimico otro);`. Hacer commit, push y pull request enseguida, para que las demás puedan compilar.
3. Crear `Acido` y `Base` con `extends Quimico`, un constructor que llame a `super(necesitaFrio)` y `@Override public boolean esCompatibleCon(Quimico otro)` según lo decidido en el Sprint 1.
4. Pensando en los motivos de derrota del Sprint 4 ("pusiste una base detrás de un ácido"), conviene que cada químico sepa decir su nombre, por ejemplo con un método que devuelva "ácido" o "base".
5. Respetar las convenciones (sección 6): sin tildes en los nombres, atributos no públicos, sin `ArrayList` ni excepciones todavía.
6. `getImagen()` (propuesto, se confirma esta semana): abstracto en `Quimico` y devuelve el nombre del dibujo, por ejemplo `"acido.png"` y `"base.png"`. Devuelve un `String` y no un `Color`: así el modelo no usa nada de Swing y la ventana no necesita ningún `if` por tipo.

TIENE QUE QUEDAR

- `Quimico.java`, `Acido.java` y `Base.java` en `src`, que compilan.

ESTÁ LISTA CUANDO

- Probar en un `main`: ácido con base da `false`, ácido con ácido da `false`, ácido con inflamable da `true` (el último, cuando Sofia2 suba `Inflamable`).
- Ningún `if` de fuera de estas clases pregunta de qué tipo es un químico.

- [ ] **Sofia2**

`Inflamable` y `Oxidante`, con `getImagen()`. Un `main` de prueba que combine los cuatro tipos de a pares (16 casos) y compare el resultado con la tabla del CLAUDE.md.

Con: IntelliJ IDEA, corriendo en la consola

Cómo hacerlo

PASOS

1. Esperar el esqueleto de `Quimico` de Sofia1 (primer día) y actualizar la rama con `main` (guía de Git).
2. Crear la rama `inflamable-oxidante` y las dos clases, igual que `Acido` y `Base`, con `getImagen()` (`"inflamable.png"` y `"oxidante.png"`).
3. Crear `PruebaCompatibilidad.java` con un `main` que arme un químico de cada tipo y pruebe los 16 pares: `a.esCompatibleCon(b)` para cada `a` y cada `b`, imprimiendo una línea por par.
4. Escribir al lado de cada línea el valor esperado según la tabla de la sección 2 y que el programa imprima "OK" o "ERROR".

TIENE QUE QUEDAR

- Dos clases nuevas y un `main` que imprime 16 líneas.

ESTÁ LISTA CUANDO

- Los 16 pares dan lo esperado: **8 en** `false` (los 4 iguales, Inflamable–Oxidante y Oxidante–Inflamable, Acido–Base y Base–Acido) y **8 en** `true` (Inflamable y Oxidante con Acido y Base, en los dos órdenes).

- [ ] **Lyssie**

`Contenedor` abstracta con array de 4 y contador: `estaLleno()`, `puedeGuardar()`, `guardar()` y `sacar()` con corrimiento, y `vaciar()` para el canje. `Heladera` y `Estante` solo dicen qué temperatura aceptan.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Crear la rama `contenedor` (guía de Git).
2. Atributos (sección 5): `protected Quimico[] quimicos = new Quimico[4];`, `protected int cantidad = 0;` y el código (`"H1"` … `"E3"`). La cantidad de lugares conviene escribirla en **un solo lugar** del código, porque el balanceo la puede bajar a 3.
3. `estaLleno()`: devuelve si `cantidad` llegó a la cantidad de lugares.
4. `puedeGuardar(Quimico nuevo)`: copiar la lógica de la sección 4.3 (lleno → no; temperatura equivocada → no; vacío → sí; si no, `ultimo.esCompatibleCon(nuevo)`).
5. `guardar(Quimico nuevo)`: si `puedeGuardar` da `true`, `quimicos[cantidad] = nuevo; cantidad++;` y devuelve `true`; si no, devuelve `false`.
6. `sacar(int posicion)`: correr hacia adelante los de atrás (del lugar `posicion` hasta `cantidad - 2`, cada uno toma el valor del siguiente); poner `null` en el último ocupado;

restar 1 a `cantidad`. **Así quedó en el código:** devuelve `boolean`, `true` si los dos que quedaron pegados son compatibles, como en el UML.

7. Recorrer siempre hasta `cantidad`, nunca hasta `.length` (sección 6).
8. `Heladera` y `Estante`: `extends Contenedor` y solo implementan el método de temperatura (la heladera acepta los que necesitan frío; el estante, los que no).
9. Agregar métodos para que la ventana pueda leer el contenido sin tocarlo: cuántos hay y qué químico hay en una posición.
10. `vaciar()`, para el canje: pone `null` en las posiciones ocupadas y deja `cantidad` en 0.

TIENE QUE QUEDAR

- Tres clases que compilan, con la cola funcionando sobre un array.

ESTÁ LISTA CUANDO

- En un `main`: guardar en una heladera tres químicos de frío compatibles entre sí, sacar el del medio y ver que el tercero pasó al lugar 2 y que `cantidad` bajó a 2.
- Un químico de frío en un estante da `false`.
- Guardar uno más en un contenedor con los 4 lugares ocupados da `false`.
- Después de `vaciar()` un contenedor lleno, `cantidad` es 0 y se puede volver a guardar.

Luz

`Deposito` con los seis contenedores (H1 a H3, E1 a E3): encuentra uno por código, le delega la operación y guarda el último motivo de error. `canjear(codigo)` para el canje. Un `main` de consola que junte todo.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Crear la rama `deposito` (guía de Git).
2. Atributos: `private Contenedor[] contenedores = new Contenedor[6];` y `private String ultimoMotivo;`.
3. Constructor: crear tres `Heladera` con códigos H1, H2, H3 y tres `Estante` con E1, E2, E3.
4. Método para buscar por código: recorrer el array y comparar con `.equals()`, nunca con `==` (sección 5). Si no lo encuentra, devuelve `null`.
5. `guardar(Quimico q, String codigo)`: busca el contenedor y le delega. Si no se pudo, deja en `ultimoMotivo` por qué. El motivo se arma preguntando en orden lo mismo que `puedeGuardar()`: si está lleno, si es la temperatura equivocada o si es incompatible con el último. En el Sprint 4 Sofia1 lo deja con el texto final.
6. `getUltimoMotivo()`: devuelve ese texto.
7. `canjear(String codigo)`: busca el contenedor; si está lleno, lo vacía con `vaciar()` y devuelve `true`; si no, devuelve `false`. Los puntos no los suma el depósito: los suma `Juego` en el Sprint 3.
8. Un `main` de consola (`PruebaDeposito`) que arme un depósito y pruebe los tres casos del criterio del sprint.

TIENE QUE QUEDAR

- `Deposito.java` y un `main` que imprime los tres casos del criterio.

ESTÁ LISTA CUANDO

- El `main` muestra: una base de frío y después un ácido de frío en H1 da `false` con su motivo; un químico de frío en E1 da `false`; sacar el del medio de un contenedor con tres corre al de atrás; canjear un contenedor lleno lo deja vacío.
- `Deposito` no tiene ningún `if` que pregunte el tipo de químico: solo delega.

- [ ] **Todas**

Cada tarea en su rama; entra a `main` después de que la revisora aprueba el pull request.

Con: GitHub Desktop y la web de GitHub

Cómo hacerlo

PASOS

1. Seguir la guía de Git, pasos 4 a 8, y mirar la tabla "Quién revisa a quién" para saber a quién pedírselo.
2. La revisora usa la lista "Qué mirar en cada revisión". Si encuentra algo, lo comenta en la línea y no aprueba hasta que se corrija.

ESTÁ LISTA CUANDO

- Todo lo que está en `main` tiene un pull request aprobado por otra, y `main` compila.

- [ ] **Todas**

Domingo 11/10: demo por consola y revisión cruzada.

Con: Meet

Cómo hacerlo

PASOS

1. Todas actualizan `main` en su computadora (guía de Git).
2. Una comparte pantalla y corre el `main` de Luz y el de Sofia2.
3. Cada una explica en voz alta una clase que no escribió.
4. Comparar el código con el diagrama: todo método nuevo que no esté en el diagrama se agrega al diagrama esa noche.

ESTÁ LISTA CUANDO

- Se cumple el criterio del sprint.

- [ ] **Todas**

Resolver las dudas que dejó el boceto, antes de que Sofia1 cierre el UML corregido (martes 6/10). El profe aprobó el UML del Sprint 1; lo nuevo del boceto todavía no está en el diagrama.

Con: WhatsApp o Meet, y el CLAUDE.md

Cómo hacerlo

PASOS

1. **1. Archivos para el top 5.** El CLAUDE.md del boceto dice que leer y escribir archivos no está en el cronograma, pero la sección 5 del CLAUDE.md del repo los pone en la clase 14 (2/11), un día después de que termina el Sprint 5. Decidir si el top 5 con `puntajes.txt` pasa al Sprint 6 o se consulta igual con el profe para hacerlo antes. Corregir la frase en el CLAUDE.md y en la guía del boceto.

2. **2. Quién llama a `siguienteTurno()`.** La sección 4.2 del CLAUDE.md del boceto dice que lo llama el listener; el UML y las secuencias dicen que la ventana llama a `juego.jugar(...)` y el `Juego` avanza solo (`siguienteTurno()` privado). Elegir una y dejar iguales el CLAUDE.md, el UML, las secuencias y las tareas de los Sprints 3 y 4.
3. **3. «Perder si piden algo que canjeaste».** En la sección 1 figura como decidido y en las secciones 2.5 y 11 como propuesto. Además, como vale cualquier químico que coincida, solo se pierde si no queda ninguno igual en otro estante. Dejar una sola versión.
4. **4. Colores por tipo y la regla 4.1.** El frasco del mostrador «del color del tipo» y las constantes ACIDO, BASE, INFLAMABLE y OXIDANTE obligarían a la ventana a preguntar el tipo con un `if`. Propuesta: el frasco también sale de `getImagen()`, y esos cuatro colores se usan solo para dibujar los PNG.
5. **5. La clase `Colores`.** No está entre las 17 clases. Decidir si cuenta (y son 18) o si se aclara que es solo una lista de constantes que no va al UML.
6. **6. Paneles con nombre de clase.** La guía del boceto nombra `PanelHud`, `PanelEscena`, `PanelFicha`, `PanelMueble` y otros, pero la sección 3 dice que las partes chicas no son clases. Ojo: dibujar el fondo con `paintComponent` obliga a que `PanelEscena` sea una clase. Alternativa: el fondo como un `JLabel` con `ImageIcon`. Corregir la guía según lo que se elija.
7. **7. `TablaPuntajes` y la ventana.** Va en el modelo pero la conoce `VentanaJuego`, que hasta ahora solo le hablaba al `Juego`. Decidir si se mantiene así (dura entre partidas) y anotar el porqué: lo pueden preguntar en la defensa.
8. **8. Detalles.** En la guía, `setIcon(q.getImagen())` pasa a `setIcon(new ImageIcon(...))`, porque `getImagen()` devuelve un `String`. Sumar `BORDE_VACIA` a la paleta. Para el motivo «canjeaste el estante E1», el modelo tiene que recordar en qué estante estaba lo canjeado. El boceto ya muestra «Volver al mostrador», que sigue pendiente para el Sprint 4.
9. Anotar cada decisión en el CLAUDE.md y avisarle a Sofia1 para que el UML corregido ya las tenga.

TIENE QUE QUEDAR

- Una decisión anotada para cada una de las ocho dudas, y el CLAUDE.md, el boceto y el UML diciendo lo mismo.

ESTÁ LISTA CUANDO

- Nadie programa el Sprint 3 con una versión distinta de estas reglas.

- [ ] Todas

Comparar el diseño con la guía del profe (tabla «Lo que tiene que poder señalarse») y sumar al UML lo que falta: una interfaz, la clase `Puntaje`, el mapa, la sobrecarga con `this()`, `static` y default.

Con: draw.io y el CLAUDE.md

Cómo hacerlo

PASOS

1. Leer juntas la tabla de la guía en este plan y marcar en cuál de las propuestas están de acuerdo.
2. **Sofia1:** la interfaz (propuesta: `Dibujable` con `getImagen()`, que implementan `Quimico` y `Visitante`). En UML va con «interface» arriba del nombre y la flecha de implementación es punteada con triángulo vacío.
3. **Lyssie:** en `Contenedor`, dos constructores (`Contenedor(codigo)` llama a `this(codigo, LUGARES)`), la constante `static final int LUGARES = 4` y `vaciar()` sin modificador

(default)`.

4. **Luz:** en `Deposito`, el mapa `HashMap<String, Contenedor>` para el Sprint 4 y la sobrecarga de un método (falta elegir cuál).
5. **Sofia2:** la clase `Puntaje` con `Comparable<Puntaje>` para el top 5, y si quieren lo opcional: `Visitante` inmutable y el enum `TipoPedido`.
6. Sumar al CLAUDE.md una sección con la tabla de la guía: es la base de la memoria técnica.

TIENE QUE QUEDAR

- El UML corregido con lo del boceto y lo de la guía, y cada clase nueva contada.

ESTÁ LISTA CUANDO

- Cada fila de la tabla de la guía dice en qué clase va, o está marcada «A consultar».

☐ **Todas**

Mandarle al profe las consultas de la guía antes del lunes 19/10 (el 12/10 es feriado).

Con: mail o el aula virtual

− **Cómo hacerlo**

PASOS

1. Preguntar: (1) en qué grupo están, si en el 1 (16/11) o en el 2 (30/11); (2) qué espera ver de «destructores», si Java no los tiene; (3) si cuenta el archivo de puntajes, la ventana Swing o las dos (Unidad 6); (4) si un hilo que anima la llegada del visitante o guarda el top 5 alcanza para «hilos»; (5) qué hay que mostrar en cada pre-entrega de los lunes.
2. Una sola manda el mensaje, en nombre del grupo.
3. Anotar las respuestas en el CLAUDE.md y en la tabla «Lo que falta decidir».

TIENE QUE QUEDAR

- Las cinco respuestas anotadas.

ESTÁ LISTA CUANDO

- Ninguna fila de la guía sigue «A consultar» después del 19/10.

CRITERIO PARA PASAR AL SPRINT 3

El `main` de consola muestra que un ácido detrás de una base devuelve `false`, que un químico de frío no entra al estante, que sacar del medio corre a los de atrás y que canjear un estante lleno lo vacía. El código coincide con el diagrama.

SPRINT 3 · 12/10 AL 18/10

0 de 10 hechas **Terminado**

El juego por consola ▲

Se juega una partida entera escribiendo en la consola. Así queda probado que el modelo anda sin Swing, y la ventana después solo se enchufa.

☐ **Todas**

En la reunión del lunes, decidir lo que falta (tabla «Lo que falta decidir»): estante lleno, fin de la partida, canje, retiros y quién dibuja los PNG. Anotar cada decisión en el CLAUDE.md.

Cómo hacerlo

PASOS

1. Decidir lo que el Sprint 3 necesita: (1) si guardar en un estante lleno es derrota o solo un aviso; (2) cuándo termina la partida si no se pierde (propuesta: se juega hasta perder); (3) el valor del canje y si canjear gasta el turno; (4) si los retiros salen de los pendientes y si se pierde apenas llega un retiro que no tenés; (5) quién dibuja y exporta los PNG.
2. Pensar cada opción con el balanceo de la sección 8: el juego ya se pierde rápido, así que una regla que haga perder más fácil lo empeora.
3. Anotar las decisiones en la sección 2 del CLAUDE.md, marcarlas en la tabla «Lo que falta decidir» y avisarle a Sofia1, que las programa en `Juego`.

TIENE QUE QUEDAR

- Una línea por decisión en la sección 2 del CLAUDE.md.

ESTÁ LISTA CUANDO

- Sofia1 sabe qué hacer en `Juego` para el estante lleno, el fin de la partida, el canje y los retiros.

Sofia1

`Juego` : turnos, pedidos, visitantes y pendientes. Cada pedido lo trae un visitante; los retiros salen de los pendientes. Si no hay pendientes, el pedido es de dejar.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Crear la rama `juego` (guía de Git).
2. Atributos: el `Deposito`, el turno, el puntaje, el visitante actual, si el pedido actual es de dejar o de retirar, qué se pide y los pendientes.
3. `Visitante` : una clase nueva con nombre e imagen, por ejemplo `new Visitante("Dra. Mora", "mora.png")` .
4. **Pedido de dejar:** elegir al azar un visitante, uno de los cuatro tipos y si necesita frío o calor, y crear ese químico. Anotarlo en los pendientes (quién lo dejó y qué dejó).
5. **Pendientes:** array con contador, igual que `Contenedor`, hasta que llegue `ArrayList` (19/10). Guardan lo que se dejó y no se retiró, incluido lo canjeado.
6. **Pedido de retirar** (si el lunes se confirma): elegir al azar uno de los pendientes; vuelve el mismo visitante y pide ese **tipo y esa temperatura**. Vale cualquier químico que coincida. Si no hay pendientes, el pedido es de dejar. Esto reemplaza «elegir entre los químicos que ya están guardados»: con el canje, el pedido puede ser algo que ya no está.
7. `canjear(codigo)` : vacía el estante a través del depósito y suma los puntos del canje. El valor va en un solo lugar del código.
8. Para comparar lo que la jugadora saca con lo pedido, la comparación de tipo tiene que estar dentro de las clases de químico (sección 4.1), no en `Juego` .
9. Para el azar, usar lo que se haya visto en clase para números aleatorios; si no se vio nada, consultarlo en el grupo antes de elegir.
10. `siguienteTurno()` : suma 1 al turno y genera el pedido nuevo. Y un método que diga si la partida terminó, según lo decidido el lunes.

TIENE QUE QUEDAR

- `Juego.java` que, sin ventana, genera pedidos y lleva el turno.

ESTÁ LISTA CUANDO

- En un `main`, 20 pedidos seguidos: aparecen los cuatro tipos, los dos de temperatura y los dos tipos de pedido.
- Sin pendientes, los 20 son de dejar.
- Nunca pide retirar algo que nadie dejó.
- Canjear un estante lleno lo vacía y suma los puntos.

Lyssie

Retiros: después de sacar del medio, revisar si los dos que quedaron pegados son compatibles. Si no, es derrota y queda el motivo. `Deposito` delega el retiro. Detectar «no lo tenías».

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Crear la rama `retiros` (guía de Git).
2. En `Deposito`, un método para retirar de un contenedor y una posición. Primero revisa que el químico de esa posición sea del tipo y la temperatura pedidos; si no, es derrota por retiro equivocado, con su motivo.
3. Si es el correcto, el contenedor lo saca con `sacar()` (ya corre a los de atrás).
4. Después del corrimiento, solo puede haber un par nuevo: el que estaba antes del hueco y el que se corrió a ocupar el hueco. Si los dos existen, revisar que el de adelante sea compatible con el de atrás con `esCompatibleCon()`. Si no lo es, es derrota y se guarda el motivo.
5. No "arreglar" el corrimiento: es la mecánica del juego (secciones 2 y 9).
6. **«No lo tenías»** (si el lunes se confirma): cuando llega un retiro y en el depósito no queda ninguno de ese tipo y esa temperatura (por ejemplo, porque se canjeó), se pierde en ese momento. Lo detecta el modelo y deja el motivo. **Al final lo programó Sofia1 en `Juego.revisarQueLoTenes()`.**
7. **Así quedó:** `Deposito.retirar(codigo, posicion, pedido)` usa `coincideConPedido` (tipo y temperatura) y además revisa que el código exista y que la casilla no esté vacía.

TIENE QUE QUEDAR

- Retirar funciona por consola y avisa cuando deja incompatibles pegados.

ESTÁ LISTA CUANDO

- Prueba: guardar ácido, inflamable y base, los tres de frío, en H1 (orden compatible) y sacar el inflamable. Quedan ácido y base pegados: tiene que ser derrota con motivo.
- Sacar el último nunca genera un par nuevo.
- Dejar un oxidante de calor en E1, llenar E1, canjearlo y pedir ese oxidante: es «no lo tenías», con motivo.

Sofia2

Partida por consola: muestra el depósito como texto y el pedido actual, y pregunta dónde guardar o qué retirar. Solo llama a `Juego` y `Deposito`; no decide ninguna regla.

Con: IntelliJ IDEA y `Scanner` para leer lo que se escribe

Cómo hacerlo

PASOS

1. Crear la rama `partida-consola` (guía de Git).
2. Un `main` en una clase de prueba (por ejemplo, `PartidaConsola`) con un ciclo que se repite hasta que la partida termine.
3. En cada vuelta: imprimir los seis contenedores en texto (por ejemplo, `H1: [Acido-frio, Base-frio, _, _]`), el turno y el pedido actual.
4. Si el pedido es de dejar, leer con `Scanner` el código del contenedor (H1…E3). Si es de retirar, leer el código y la posición.
5. Un comando para canjear (por ejemplo, escribir `canjear E1`) que llama a `juego.canjear(...)`.
6. Llamar al método de `Deposito` que corresponda. Si da `false`, imprimir `getUltimoMotivo()` y terminar; si da `true`, `juego.siguienteTurno()`.
7. Revisar que esta clase no tenga ningún `if` sobre el tipo de químico ni ninguna regla: solo pregunta y muestra. Es la misma regla que después cumple la ventana.

TIENE QUE QUEDAR

- Una partida que se juega escribiendo en la consola.

ESTÁ LISTA CUANDO

- Se puede perder por cada motivo (dos iguales, explosión, temperatura, retiro equivocado, retiro que deja incompatibles, no lo tenías) y el mensaje dice cuál fue.
- Se puede canjear un estante lleno desde la consola.

Luz

Aprender Swing con un proyecto de práctica aparte: `CardLayout` con dos pantallas, una escena con `setLayout(null)` e `ImageIcon`, y un estante con `BoxLayout`. Escrito a mano, sin editor visual.

Con: IntelliJ IDEA, en un proyecto separado del juego

Cómo hacerlo

PASOS

1. Crear un proyecto nuevo en IntelliJ, fuera del repositorio del juego (guía de IntelliJ).
2. Una clase que extiende `JFrame`, con título, tamaño y que se cierre al cerrar la ventana.
3. **Dos pantallas con `CardLayout`:** un botón «Ir al depósito» pasa de una a otra con `cartas.show(...)`, como en la guía del boceto.
4. **Una escena con `setLayout(null)`:** un `JLabel` con `ImageIcon` ubicado con `setBounds(...)`, como el visitante del mostrador.
5. **Un estante con `BoxLayout`:** el código, cuatro `JButton` de casilla y un botón Canjear que se muestra o se esconde con `setVisible(...)`.
6. Un clic en una casilla cambia un texto de la ventana (por ejemplo, "hiciste clic en H2, casilla 3"). Para escuchar el clic sirven las clases internas de la clase 10.
7. Todo escrito a mano: nada de diseñadores visuales, porque el código que generan no se puede explicar en la defensa.

TIENE QUE QUEDAR

- Un prototipo con dos pantallas, una escena con un dibujo y un estante que reacciona a los clics.

ESTÁ LISTA CUANDO

- Luz puede explicar cada línea del prototipo y hacerlo de nuevo sin mirar el tutorial.

Luz

Viernes 16/10: enseñarle a Sofia2 lo que aprendió de Swing, con su prototipo.

Con: Meet

Cómo hacerlo

PASOS

1. Compartir pantalla con el prototipo.
2. Mostrar tres cosas: cómo se arma la ventana, cómo se ubican los paneles y cómo se escucha un clic.
3. Sofia2 cambia algo en vivo (por ejemplo, el texto que aparece al hacer clic) para comprobar que lo entendió.

ESTÁ LISTA CUANDO

- Sofia2 puede agregar un listener a un botón sin ayuda.

Todas

Domingo 18/10: cada una juega una partida completa por consola y revisa el código de otra.

Con: GitHub y Meet

Cómo hacerlo

PASOS

1. Todas actualizan `main` (guía de Git) y corren `PartidaConsola`.
2. Cada una juega una partida entera y anota en qué turno perdió y por qué, en la tabla compartida que después usa Lyssie para el balanceo.
3. Revisión cruzada según la tabla "Quién revisa a quién".

ESTÁ LISTA CUANDO

- Se cumple el criterio del sprint.

Todas

Exportar los PNG del boceto: personajes, íconos de químicos y fondo del mostrador. La dueña se decide el lunes 12/10.

Con: un editor de pixel art gratuito

Cómo hacerlo

PASOS

1. Personajes: están dibujados en el boceto a 32 × 45 px y exportados x4 (128 × 180) para el mostrador y x2 (64 × 90) para el depósito.
2. Íconos de químicos: se dibujan a 8 × 8 y se exportan x3 (24 px). Todavía faltan.
3. Siempre se exportan ya agrandados: agrandarlos desde Java los deja borrosos.
4. Nombres en minúscula, sin tildes y con el tamaño al final, en `src/img/` : `lyss_x4.png`, `mora_x2.png`, `heladera_x2.png`, `escena_explota.png` … Los del boceto ya están en el pull request #7. Faltan `acido.png`, `base.png`, `inflamable.png`, `oxidante.png` y `fondo_mostrador.png`.

5. Si lleva más tiempo de lo previsto, se sigue con rectángulos de color (tabla de riesgos).

TIENE QUE QUEDAR

- Los PNG en `src/img/`, con los nombres de la lista.

ESTÁ LISTA CUANDO

- La práctica de Swing de Luz carga uno con `ImageIcon` y se ve nítido.

- [ ] **Sofia1**

Temas de la guía en `Juego` y `Visitante`: casteo al elegir al azar y, si lo eligieron, `Visitante` inmutable y el enum `TipoPedido`.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Casteo: `int tipo = (int) (Math.random() * 4);` para elegir uno de los cuatro tipos. Anotar en la memoria que es un casteo de `double` a `int`.
2. Opcional, `Visitante` inmutable: atributos `private final`, se cargan solo en el constructor y no hay setters.
3. Opcional, enum: `public enum TipoPedido { DEJAR, RETIRAR }` en lugar del `boolean retiro`.

TIENE QUE QUEDAR

- Cada tema anotado en la tabla de la guía con la clase y el método.

ESTÁ LISTA CUANDO

- Otra integrante encuentra el casteo en el código mirando solo la tabla.

- [ ] **Todas**

Lunes 19/10: pre-entrega sin nota. Mostrarle al profe la partida por consola y el UML corregido.

Con: GitHub y draw.io

Cómo hacerlo

PASOS

1. Llevar el UML con lo del boceto y lo de la guía, y la partida por consola andando.
2. Hacerle las consultas que todavía no haya contestado por mail.
3. Una anota lo que diga el profe en una lista numerada en el grupo.

TIENE QUE QUEDAR

- La lista de comentarios del profe.

ESTÁ LISTA CUANDO

- Cada comentario tiene dueña y sprint.

CRITERIO PARA PASAR AL SPRINT 4

Se juega una partida de principio a fin por consola, con dejar, retirar y canjear.

SPRINT 4 · 19/10 AL 25/10

0 de 14 hechas Estamos acá

La ventana y pre-entrega ▲

Se juega una partida entera en la ventana, aunque sea fea. La lógica sigue en el modelo: la ventana solo pregunta y muestra.

- [ ] Luz

`VentanaJuego` con `CardLayout`, `PanelMostrador`, `PanelDeposito` y `PanelContenedor` como estante con Canjear, a partir de su práctica.

Con: IntelliJ IDEA, Swing a mano

− Cómo hacerlo

PASOS

1. Crear la rama `ventana` (guía de Git).
2. `PanelContenedor`: extiende `JPanel` y es **un estante en fila**: recibe su `Contenedor` en el constructor y muestra el código, las cuatro casillas y el botón Canjear. Un método `actualizar()` vuelve a leer el contenedor y hace `canjear.setVisible(contenedor.estaLleno())`.
3. `PanelMostrador`: turno y puntaje arriba, el visitante con su globo en el medio y abajo la ficha del pedido y el botón «Ir al depósito». Aunque sea con rectángulos.
4. `PanelDeposito`: lo que llevás o lo que te piden arriba, la heladera con H1–H3 y los estantes E1–E3 (seis `PanelContenedor`) en el medio y el mensaje abajo.
5. `VentanaJuego`: extiende `JFrame`, de 1024 × 720, y cambia de pantalla con `CardLayout` (`"mostrador"` y `"deposito"`; reglas y final llegan en el Sprint 5).
6. `actualizarPantalla()`: pide a cada panel que se actualice y vuelve a escribir el pedido, el turno y el mensaje.
7. Un `main` nuevo que cree el `Juego` y abra la ventana.
8. Las imágenes se cargan desde `src/img/` con `new ImageIcon(getClass().getResource("/img/mora_x4.png"))`, no con la ruta suelta de la práctica: así funcionan en cualquier computadora.

TIENE QUE QUEDAR

- La ventana muestra el mostrador y el depósito con el estado real del modelo.

ESTÁ LISTA CUANDO

- Si se guarda un químico por código (desde un `main` de prueba) y se llama `actualizarPantalla()`, aparece en la casilla correcta.

- [ ] Sofia2

Conectar los clics con el modelo: ir al depósito, casillas, canjear y volver. Clic en un estante guarda y clic en una casilla retira. Después de cada jugada, `actualizarPantalla()`. Trabaja de a dos con Luz.

Con: IntelliJ IDEA; para los listeners sirven las clases internas de la clase 10

− Cómo hacerlo

PASOS

1. Trabajar en la misma rama que Luz, coordinando quién edita qué archivo para no pisarse.

2. En cada listener: llamar a `juego.jugar(codigo, posicion)` (o a `juego.canjear(codigo)`); si da `false`, `mostrarMensaje(juego.getMotivo())`; al final, `actualizarPantalla()`. El turno lo avanza `Juego` solo: la ventana nunca llama a `siguienteTurno()` ni le habla a `Deposito`.
3. Clic en un contenedor, cuando el pedido es de dejar (`!juego.esRetiro()`): `juego.jugar(codigo, 0)`.
4. Clic en una casilla, cuando el pedido es de retirar: `juego.jugar(codigo, posicion)`, con la posición empezando en 0.
5. «Ir al depósito»: `cartas.show(..., "deposito")`. Si la jugada salió bien, volver al mostrador con `cartas.show(..., "mostrador")`; si no, mostrar el motivo.
6. Canjear: llama al modelo, que decide si se puede y cuánto suma. La ventana no decide nada: solo muestra el botón cuando `estaLleno()` da `true`.
7. Volver al mostrador sin jugar: solo si el grupo lo decide en este sprint.
8. Cuando se pierde, desactivar los clics para que no se pueda seguir jugando.

TIENE QUE QUEDAR

- Se juega con el mouse, sin consola.

ESTÁ LISTA CUANDO

- Ningún listener tiene un `if` que decida una regla: la única decisión es qué mostrar según el `true` o `false` del modelo.

☐ **Sofia1**

Motivos de derrota exactos, armados en el modelo, como "Pusiste una base detrás de un ácido en H2". Cubrir cada forma de perder: dos iguales, explosión, temperatura equivocada, retiro equivocado, retiro que deja incompatibles juntos y «no lo tenías».

Con: IntelliJ IDEA, probando por consola

− Cómo hacerlo

PASOS

1. Crear la rama `motivos` (guía de Git).
2. Listar todas las formas de perder: dos iguales, explosión (Inflamable–Oxidante o Acido–Base), temperatura equivocada, retiro equivocado (no era el tipo o la temperatura pedidos), retiro que deja incompatibles pegados, «no lo tenías» y, si el lunes 12/10 se decidió que es derrota, estante lleno.
3. Para cada una, un título y un motivo con los datos concretos, como en la tabla de «Reglas del juego»: qué químico, detrás de cuál y en qué estante, con su código como se ve en pantalla (H2, E1). Ya no «la heladera 2»: ahora hay una sola heladera. Los nombres los da cada químico.
4. Todo el texto se arma en el modelo; la ventana solo lo muestra.
5. Probar cada forma de perder por consola.

TIENE QUE QUEDAR

- Un mensaje distinto y concreto para cada forma de perder.

ESTÁ LISTA CUANDO

- Leyendo solo el mensaje, se entiende qué pasó y dónde, sin mirar el depósito.

☐ **Lyssie**

Pasar `Contenedor` de array con contador a `ArrayList` (clase 12, 19/10), en una rama aparte. Anotar el antes y el después: `remove()` ya hace el corrimiento solo.

Con: IntelliJ IDEA y GitHub Desktop

Cómo hacerlo

PASOS

1. Esperar a que la clase 12 (19/10) dé `ArrayList` y actualizar la sección 5 del CLAUDE.md.
2. Crear la rama `arraylist` (guía de Git).
3. Cambiar el array y el contador por un `ArrayList<Quimico>`: `cantidad` pasa a ser `size()`, guardar es `add()`, sacar es `remove(posicion)` (corre solo a los de atrás) y leer es `get(i)`.
4. Mantener un solo lugar para la cantidad de lugares.
5. Anotar en un archivo de texto o en el CLAUDE.md el antes y el después: cuántas líneas tenía `sacar()` y cuántas tiene ahora. Sirve para la defensa.
6. Correr las pruebas del Sprint 2 y una partida por consola antes del pull request. Si algo se rompe, se descarta la rama (tabla de riesgos).

TIENE QUE QUEDAR

- `Contenedor` con `ArrayList`, sin cambios en las otras clases.

ESTÁ LISTA CUANDO

- Las pruebas del Sprint 2 dan igual que antes.
- Nada fuera de `Contenedor` tuvo que cambiar.

- [ ] Lyssie

Primer balanceo con la partida por consola: cada una juega 5 partidas y anota en qué turno perdió, cuántos canjes hizo y si perdió por «no lo tenías». Si la mayoría termina antes del turno 10, ajustar en el orden de la sección 8.

Con: una tabla compartida

Cómo hacerlo

PASOS

1. Crear una planilla compartida en Google Drive con columnas: jugadora, partida (1 a 5), turno en que perdió, motivo, cuántos canjes hubo y si se perdió por «no lo tenías».
2. Pedirle a cada una que juegue 5 partidas por consola y complete su fila.
3. Contar cuántas de las 20 partidas terminaron antes del turno 10.
4. Si son la mayoría, aplicar el primer ajuste de la sección 8 (contenedores de 3 lugares), volver a jugar y recién después pasar al siguiente. Un cambio por vez.
5. No inventar reglas nuevas: solo ajustar números (sección 8). El valor del canje es una perilla más, y se ajusta igual: de a una.

TIENE QUE QUEDAR

- La planilla completa y, si hizo falta, un ajuste aplicado.

ESTÁ LISTA CUANDO

- Hay 20 partidas anotadas y la decisión (ajustar o no) está escrita en el CLAUDE.md.

- [ ] Todas

Sábado 24/10: congelar el código de la pre-entrega. Domingo 25/10: cada una juega en la ventana, se compara el UML con el código y se arma lo que se entrega el lunes 26/10.

Con: GitHub y Meet

Cómo hacerlo

PASOS

1. **Antes del sábado:** preguntarle al profe qué hay que entregar el 26/10 y en qué formato (link al repositorio, un zip, el UML en PDF). Anotarlo en el CLAUDE.md.
2. **Sábado:** se mergean los últimos pull requests aprobados y desde ahí no entra nada más a `main` hasta el lunes, salvo un arreglo de error.
3. **Domingo:** cada una juega una partida en la ventana. Se abre el diagrama al lado del código y se anota qué no coincide.
4. Armar lo que pidió el profe.

TIENE QUE QUEDAR

- La entrega armada tal como la pidió el profe.

ESTÁ LISTA CUANDO

- Se cumple el criterio de la pre-entrega.

- [ ] **Luz**

`HashMap<String, Contenedor>` en `Deposito` para buscar por código (clase 12, 19/10), y for-each donde se recorre una colección.

Con: IntelliJ IDEA y GitHub Desktop

Cómo hacerlo

PASOS

1. Crear la rama `mapa-deposito` (guía de Git).
2. Cambiar el array de seis por `HashMap<String, Contenedor>`: se carga en el constructor con `put("H1", ...)` y buscar pasa a ser `get(codigo)`.
3. Donde se recorran colecciones (pendientes, paneles), usar for-each.
4. Anotar el antes y el después para la memoria, igual que Lyssie con `ArrayList`.

TIENE QUE QUEDAR

- `Deposito` con un mapa, sin cambios fuera de esa clase.

ESTÁ LISTA CUANDO

- Las pruebas del Sprint 2 y la partida por consola dan igual que antes.

- [ ] **Sofia2**

Viene del Sprint 3. Partida por consola (`PartidaConsola`): es el criterio del Sprint 3 y todavía no está. Ahora que `Juego` está en `main` , se puede hacer y probar directo.

Con: IntelliJ IDEA y `Scanner`

Cómo hacerlo

PASOS

1. Crear la rama `partida-consola` desde `main` actualizado (guía de Git).

2. Un ciclo que se repite mientras `!juego.estaTerminado()`.
3. En cada vuelta: imprimir los seis contenedores con `juego.getContenedor("H1")`... (`getCantidad()` y `getQuimico(i).getNombre()`), el turno, el puntaje y el pedido (`getVisitante().getNombre()`, `getTipoPedido()`, `getPedido().getNombre()` y si necesita frío).
4. Leer con `Scanner` el código del contenedor y, si es un retiro, la posición. Llamar a `juego.jugar(codigo, posicion)`.
5. Un comando `canjear E1` que llama a `juego.canjear("E1")`.
6. Si `jugar` da `false`, imprimir `juego.getMotivo()`. No llamar a `siguienteTurno()`: lo hace `Juego` solo.
7. Mirar `PruebaJuego` de Sofia1: ya recorre los contenedores y arma el texto del pedido.

TIENE QUE QUEDAR

- Una partida que se juega escribiendo en la consola.

ESTÁ LISTA CUANDO

- Se puede perder por cada motivo y el mensaje dice cuál fue.
- Se puede canjear un estante lleno desde la consola.

☐ **Todas**

Actualizar el UML con lo que ya está en `main`. El diagrama del repositorio es del 5/10 y el profe lo compara con el código.

Con: draw.io al lado de IntelliJ IDEA

− **Cómo hacerlo**

PASOS

1. Cada una actualiza las clases que programó (guía de draw.io).
2. **Sofia1:** `Juego` con sus atributos y métodos reales (`canjear`, `getVisitante`, `getTipoPedido`, `getCantidadPendientes`, `PUNTOS_CANJE`), `Visitante`, el enum `TipoPedido` y `coincideConPedido` en `Quimico`.
3. **Lyssie:** `Contenedor` con `vaciar()`, `puedeGuardar`, `sacar(posicion): boolean` y `LUGARES`.
4. **Luz:** `Deposito` con `canjear`, `retirar`, `buscar` público y el último motivo.
5. **Sofia2:** revisar que las dos secuencias usen `juego.jugar(...)` y `juego.getMotivo()`.
6. Exportar el diagrama y subir el SVG nuevo al repositorio.

TIENE QUE QUEDAR

- El diagrama coincide con el código de `main`.

ESTÁ LISTA CUANDO

- Otra del grupo elige tres métodos al azar del código y los encuentra en el diagrama con el mismo nombre.

☐ **Sofia1** **Sofia2**

`getImagen()` en los químicos: abstracto en `Quimico` y cada tipo devuelve el nombre de su dibujo. La ventana del Sprint 5 lo necesita para no preguntar el tipo con un `if`.

Con: IntelliJ IDEA

− **Cómo hacerlo**

PASOS

1. **Sofia1:** `public abstract String getImagen();` en `Quimico`, y `"acido.png"` y `"base.png"` en `Acido` y `Base`.
2. **Sofia2:** `"inflamable.png"` y `"oxidante.png"` en `Inflamable` y `Oxidante`.
3. Devuelve un `String` y no un `Color`: el modelo no usa nada de Swing.
4. Los PNG de los químicos todavía no existen: se dibujan a 8 × 8 y se exportan x3 (24 px) en `src/img/`.

TIENE QUE QUEDAR

- Los cuatro químicos dicen su dibujo.

ESTÁ LISTA CUANDO

- Ningún `if` fuera de las clases de químico pregunta de qué tipo es.

Sofia1

Visitantes del boceto en `Juego`: los seis (las otras tres del equipo, la Dra. Mora, el Dr. Bigotes y el Ing. Paco) y la encargada que se elige al empezar.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Usar los nombres nuevos de los PNG: `new Visitante("Dra. Mora", "mora_x4.png")`, `"bigotes_x4.png"`, `"paco_x4.png"`, `"lyss_x4.png"`, `"luz_x4.png"`, `"sofi1_x4.png"` y `"sofi2_x4.png"`.
2. La encargada no visita: si atiende Luz, las visitantes son las otras tres más las tres del laboratorio.
3. Decidir con el grupo cómo se elige la encargada al empezar (por ejemplo, un parámetro del constructor de `Juego`).

TIENE QUE QUEDAR

- Seis visitantes y una encargada.

ESTÁ LISTA CUANDO

- En 20 pedidos seguidos nunca aparece la encargada como visitante.

Lyssie

Lo que quedó de la guía en `Contenedor`: los dos constructores unidos con `this(...)`, `vaciar()` sin modificador (default) y `@Override` en `Heladera` y `Estante`.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. `Contenedor(String codigo)` llama a `this(codigo, LUGARES)`, y `Contenedor(String codigo, int lugares)` arma el array con esa cantidad.
2. Sacarle el `public` a `vaciar()`: solo lo usa `Deposito`, que está en el mismo paquete.
3. Agregar `@Override` arriba de `aceptaTemperatura` en `Heladera` y `Estante`.

4. En `sacar`, si la posición no está ocupada, no hacer nada. `retirar` ya lo revisa antes, pero así `Contenedor` se cuida solo.

TIENE QUE QUEDAR

- `Contenedor` con los temas de la guía que le tocaban.

ESTÁ LISTA CUANDO

- Las pruebas del Sprint 2 y `PruebaJuego` dan igual que antes.

- [ ] **Luz**

Sobrecarga de un método: `guardar(Quimico q)` en `Deposito`, al lado de `guardar(q, codigo)`.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Recorre los contenedores y lo guarda en el primero donde entre, reutilizando `guardar(q, codigo)`.
2. Si no entra en ninguno, devuelve `false` con un motivo.
3. Anotarlo en la tabla de la guía: es el ejemplo de sobrecarga de un método.

TIENE QUE QUEDAR

- Dos métodos `guardar` en `Deposito` con distintos parámetros.

ESTÁ LISTA CUANDO

- Un `main` de prueba usa los dos.

- [ ] **Todas**

Revisar y mergear el pull request #7: los PNG del boceto en `src/img/` con nombres unificados.

Con: GitHub

Cómo hacerlo

PASOS

1. Abrir el pull request en GitHub y mirar la lista de nombres.
2. Si están de acuerdo con los nombres, mergearlo. Después, `Ctrl+T` en `main`.
3. Avisarle a Sofia1 que `Juego` pasa a usar `"mora_x4.png"` y `"bigotes_x4.png"`.

TIENE QUE QUEDAR

- Los dibujos en `main`, en `src/img/`.

ESTÁ LISTA CUANDO

- La ventana carga `mora_x4.png` con `getResource` y se ve nítido.

CRITERIO DE LA PRE-ENTREGA (LUNES 26/10)

Se juega una partida yendo del mostrador al depósito, aunque sea con rectángulos. Ningún listener decide una regla.

SPRINT 5 · 26/10 AL 1/11

0 de 7 hechas 26/10 al 1/11

Que se entienda y excepciones ▲

Alguien que nunca vio el juego lo juega sin explicación y entiende por qué perdió. Con este sprint el juego queda completo.

- [ ] **Sofia2**

Puntaje, pantalla final y top 5 ordenado con `Comparable`. El canje ya suma; decidir qué más suma. El top 5 se guarda en archivo en el Sprint 6.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Proponer en el grupo qué suma puntos (por ejemplo, cada jugada válida) y, si en el balanceo se decidió permitir rechazar entregas, cuánto resta rechazar (sección 8). Decidirlo entre las cuatro y anotarlo en el CLAUDE.md.
2. El puntaje lo lleva `Juego` (sección 3); la ventana solo lo muestra.
3. Pantalla final (`PanelFinal`), ocupando toda la ventana: dibujo, título, qué pasó, turno, puntaje, «Jugar de nuevo» (crea un `Juego` nuevo) y «Ver reglas».
4. **Top 5 en memoria:** una clase `Puntaje` (iniciales, puntos y turno) que implementa `Comparable<Puntaje>`; `TablaPuntajes` guarda una lista, la ordena con `Collections.sort(...)` y se queda con los 5 primeros. Si la partida entra, se piden las iniciales.
5. Por ahora el top 5 dura mientras el juego está abierto. Leerlo y guardarlo en `puntajes.txt` es del Sprint 6, cuando la clase 14 (2/11) dé archivos.

TIENE QUE QUEDAR

- Puntaje visible durante la partida, una pantalla final y un top 5 ordenado.

ESTÁ LISTA CUANDO

- Jugar de nuevo arranca con el depósito vacío y el puntaje en 0.

- [ ] **Luz**

Aplicar el boceto (sección 7 del CLAUDE.md del boceto): paleta, fuentes, dibujos y estados de casilla. La pantalla de reglas ya está diseñada.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Antes de crear la ventana: `UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName())`. Con el aspecto de Windows o Mac, los botones ignoran `setBackground`.
2. **Paleta:** constantes en una clase `Colores`, con los valores del boceto. En el resto del código no se escriben números de color.
3. **Fuentes:** Pixelify Sans y Nunito ExtraBold desde `src/fuentes/` con `Font.createFont`; si falla, una fuente común y el juego sigue.

4. **Dibujos:** la ventana carga cada químico con `new ImageIcon(getClass().getResource("/img/" + q.getImagen()))`, sin ningún `if` por tipo.
5. **Estados de casilla** (cada una es un `JButton`): vacía, próxima libre, ocupada, «último», mouse encima (solo cambia el borde) y partida terminada. En cada casilla: `setOpaque(true)`, `setFocusPainted(false)` y `setBorder(...)`.
6. Pantalla de reglas: armarla como en el boceto.
7. Si Swing se complica, esto es lo primero que se recorta (tabla de riesgos).

TIENE QUE QUEDAR

- La ventana se entiende sin explicación.

ESTÁ LISTA CUANDO

- Alguien que no conoce el juego puede decir, mirando la pantalla, qué le piden y dónde lo puede poner.

Sofia1

Una excepción propia para las jugadas inválidas (clase 13, 26/10), con el motivo como mensaje. Cubre también el canje. Reemplaza el `boolean` más `getUltimoMotivo()`; la ventana la atrapa y muestra el mensaje. Anotar el antes y el después.

Con: IntelliJ IDEA, en una rama aparte

− Cómo hacerlo

PASOS

1. Esperar a que la clase 13 (26/10) dé excepciones propias y actualizar la sección 5 del CLAUDE.md.
2. Crear la rama `excepcion` (guía de Git).
3. Crear una clase de excepción propia (por ejemplo, `JugadaInvalidaException`) que reciba el motivo y se lo pase a `super(motivo)`, como se vea en la clase 13.
4. En el modelo, donde hoy se devuelve `false` y se guarda el motivo, lanzar la excepción con el motivo.
5. En la ventana y en la partida por consola, envolver la jugada en `try/catch` y mostrar el mensaje de la excepción.
6. Anotar el antes y el después para la defensa. Si rompe algo, se descarta la rama (tabla de riesgos).

TIENE QUE QUEDAR

- Jugadas inválidas informadas con una excepción propia.

ESTÁ LISTA CUANDO

- Cada forma de perder del Sprint 4 sigue mostrando su mensaje, ahora desde la excepción.

Lyssie

Balanceo final en la ventana: cada una juega 10 partidas y anota el turno en que perdió. La cantidad de lugares y el valor del canje se cambian en un solo lugar del código.

Con: una tabla compartida

− Cómo hacerlo

PASOS

1. Usar la misma planilla del Sprint 4, con una pestaña nueva.
2. Cada una juega 10 partidas en la ventana y anota turno y motivo.
3. Si la mayoría de las 40 termina antes del turno 10, seguir con el ajuste siguiente de la sección 8, uno por vez.
4. Comprobar que la cantidad de lugares está en un solo lugar del código: cambiar ese número tiene que alcanzar.

TIENE QUE QUEDAR

- 40 partidas anotadas y los ajustes que hicieron falta.

ESTÁ LISTA CUANDO

- La decisión final de balanceo está escrita en el CLAUDE.md.

☐ Lyssie

Conseguir a alguien de afuera del grupo para probar el juego el domingo.

Con: Meet con pantalla compartida, o en persona

− Cómo hacerlo

PASOS

1. Buscar a alguien que no haya visto el juego ni el código.
2. Acordar el domingo 1/11 y cómo (Meet o en persona).
3. Preparar la hoja de observación: dónde se traba, qué pregunta, si entendió el canje y, al final, si puede explicar por qué perdió.

ESTÁ LISTA CUANDO

- Hay una persona confirmada para el domingo.

☐ Todas

Domingo 1/11: la persona de afuera juega sin ayuda, y revisión cruzada.

Con: GitHub y Meet

− Cómo hacerlo

PASOS

1. Nadie le explica nada: solo se le dice "jugá".
2. Una anota dónde se traba y qué pregunta.
3. Cuando pierde, preguntarle: "¿por qué perdiste?" y anotar la respuesta con sus palabras.
4. Si no puede explicarlo, mejorar el mensaje de derrota esa misma semana.
5. Después, revisión cruzada según la tabla.

ESTÁ LISTA CUANDO

- Se cumple el criterio del sprint.

☐ Todas

Lunes 2/11: pre-entrega sin nota con el juego completo en la ventana. El lunes siguiente (9/11) es feriado.

Con: GitHub

Cómo hacerlo

PASOS

1. Llevar el juego jugable de punta a punta, con la pantalla final y el top 5.
2. Mostrar la tabla de la guía con lo que ya se puede señalar en el código y lo que falta (archivos, paquetes, hilo y memoria).
3. Anotar lo que diga el profe y repartirlo para los Sprints 6 y 7.

TIENE QUE QUEDAR

- La lista de comentarios del profe, repartida.

ESTÁ LISTA CUANDO

- Cada comentario tiene dueña y sprint.

CRITERIO PARA PASAR AL SPRINT 6

Esa persona explica con sus palabras por qué perdió. Si no puede, el mensaje de derrota todavía no alcanza.

SPRINT 6 · 2/11 AL 8/11

0 de 7 hechas 2/11 al 8/11

UML final, archivos, hilo y memoria ▲

Se suma lo último que pide la guía (archivo, paquetes e hilo), el UML se pone al día con el código y se escribe el borrador de la memoria técnica.

- [ ] Sofia1 Lyssie Luz

Cada una actualiza la parte del diagrama de clases que dibujó en el Sprint 1, comparándola clase por clase con el código final. Incluye `ArrayList`, el mapa, la interfaz, `Puntaje` y la excepción nueva.

Con: draw.io al lado de IntelliJ IDEA

Cómo hacerlo

PASOS

1. Abrir el diagrama y el código lado a lado.
2. Para cada clase de su parte, revisar uno por uno: atributos (nombre, tipo y visibilidad), métodos (nombre, parámetros y lo que devuelven) y relaciones.
3. Agregar lo que se sumó después del Sprint 1: `ArrayList`, `HashMap`, la interfaz, `Puntaje`, la excepción propia y cualquier método nuevo.
4. Actualizar también el diagrama unido. El UML de vista muestra las 5 clases y el `CardLayout`.

TIENE QUE QUEDAR

- El diagrama coincide con el código, clase por clase.

ESTÁ LISTA CUANDO

- Otra del grupo elige tres métodos al azar del código y los encuentra en el diagrama con el mismo nombre.

Sofia2

Top 5 en `puntajes.txt` (clase 14, 2/11), con `try`, `catch` y `finally`.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Crear la rama `puntajes-archivo` (guía de Git).
2. Leer con `Scanner` y `File`, una línea por puesto (`LUZ;340;31`), separando con `split(";")` y convirtiendo con `Integer.parseInt(...)`.
3. Escribir con `PrintWriter`. Cerrar el archivo en un `finally`.
4. Si el archivo no existe (`FileNotFoundException`) o una línea está rota (`NumberFormatException`), la tabla arranca vacía o saltea esa línea, y el juego sigue.
5. Solo `TablaPuntajes` toca archivos.

TIENE QUE QUEDAR

- El top 5 se conserva al cerrar y abrir el juego.

ESTÁ LISTA CUANDO

- Borrar `puntajes.txt` o romper una línea a mano no hace caer el juego.

Sofia2

Actualizar las dos secuencias para que muestren la excepción en lugar del `boolean`.

Con: draw.io

Cómo hacerlo

PASOS

1. En guardar y en retirar, reemplazar la respuesta `false` más `getUltimoMotivo()` por la excepción que sube hasta la ventana.
2. Mostrar dónde se atrapa (`try/catch` en la ventana) y que la ventana solo muestra el mensaje.

TIENE QUE QUEDAR

- Dos secuencias al día con el código final.

ESTÁ LISTA CUANDO

- Siguiendo las flechas se ve de dónde sale la excepción y quién la atrapa.

Luz

Paquetes `modelo` y `vista` (clase 14, 2/11) y el hilo que pide la guía.

Con: IntelliJ IDEA

Cómo hacerlo

PASOS

1. Crear la rama `paquetes-hilo` (guía de Git).
2. Crear los paquetes `modelo` y `vista` y mover las clases con la opción de mover de IntelliJ, que arregla los imports sola.
3. Comprobar que el paquete `modelo` no importa nada de `java.awt` ni de `javax.swing`.

4. **Hilo:** lo que se haya decidido con el profe (por ejemplo, animar en un hilo aparte la llegada del visitante o la explosión). Se aprende por cuenta propia: hilos llegan en la clase 18 (30/11). Lo que toca la ventana desde el hilo se hace con `SwingUtilities.invokeLater(...)`.
5. El hilo no cambia ninguna regla del juego.

TIENE QUE QUEDAR

- Dos paquetes y un hilo que se puede señalar en el código.

ESTÁ LISTA CUANDO

- El proyecto compila y se juega una partida igual que antes.

- [ ] **Sofia1**

Borrador de la memoria técnica: el diseño en una página y la tabla de la guía con la clase y el método de cada tema.

Con: Google Docs

− **Cómo hacerlo**

PASOS

1. Partir de la tabla «Lo que tiene que poder señalarse» de este plan.
2. Para cada fila, poner la clase y el método exactos del código final.
3. Nombrar explícitamente el patrón GRASP y el principio SOLID elegidos, con el porqué.
4. Sumar el antes y el después de `ArrayList`, el mapa y la excepción.

TIENE QUE QUEDAR

- Un documento compartido con las cuatro.

ESTÁ LISTA CUANDO

- Ninguna fila de la tabla queda sin clase y método.

- [ ] **Lyssie**

Simulacro del quinto tipo (imaginario): en una rama aparte que después se borra, las cuatro inventan un tipo de químico nuevo y cuentan qué clases hubo que tocar. Si hubo que modificar alguna existente, revisar el diseño.

Con: IntelliJ IDEA y GitHub Desktop; la rama se borra al terminar

− **Cómo hacerlo**

PASOS

1. Crear la rama `simulacro-quinto` (guía de Git). Esta rama no se mergea nunca: el CLAUDE.md prohíbe agregar tipos de químico al juego (sección 9).
2. Entre las cuatro, crear una clase nueva que extienda `Quimico` con su compatibilidad y su `getImagen()`.
3. Anotar cada archivo que haya que tocar para que el químico nuevo funcione. Mirar también si `Juego` necesita cambiar para que aparezca en los pedidos al azar, y discutir si eso cuenta como modificar una clase existente.
4. Si hubo que tocar alguna clase de químico que ya existía, revisar la decisión del Sprint 1.
5. Borrar la rama al terminar.

TIENE QUE QUEDAR

- Una lista de las clases que hubo que tocar, para contestar la pregunta 2 de la defensa.

ESTÁ LISTA CUANDO

- Las cuatro pueden responder la pregunta del quinto tipo con lo que pasó en el simulacro.

Todas

Domingo 8/11: revisión cruzada y repaso de la tabla de la guía.

Con: GitHub y Meet

Cómo hacerlo

PASOS

1. Revisión cruzada según la tabla.
2. Recorrer la tabla de la guía fila por fila y abrir en el código cada lugar que señala.

ESTÁ LISTA CUANDO

- Se cumple el criterio del sprint.

CRITERIO PARA PASAR AL SPRINT 7

Cada tema de la guía se puede señalar en el código, incluidos el archivo, los paquetes y el hilo. El UML coincide con el código.

SPRINT 7 · 9/11 AL 15/11

0 de 6 hechas 9/11 al 15/11

Defensa

No se agregan funciones. Se termina la memoria, se congela el código y las cuatro practican defenderlo. Si les toca el Grupo 2 (30/11), las dos semanas que sobran son colchón.

Luz

Memoria final: el diseño y la tabla de temas de la guía, revisada contra el código congelado.

Con: Google Docs

Cómo hacerlo

PASOS

1. Partir del borrador de Sofia1.
2. Abrir cada clase y método de la tabla en el código y confirmar que sigue ahí con el mismo nombre.
3. Sumar los diagramas exportados.
4. Exportar a PDF.

TIENE QUE QUEDAR

- La memoria en PDF.

ESTÁ LISTA CUANDO

- Cualquiera del grupo puede abrir el código en cada lugar que señala la memoria.

☐ **Sofia1**

Organizar el ensayo con las nueve preguntas: las seis del CLAUDE.md y las tres del boceto. Cada una contesta todas, no solo las de su parte.

Con: Meet

− **Cómo hacerlo**

PASOS

1. Las seis preguntas de la sección 10: (1) ¿por qué la compatibilidad se resuelve con polimorfismo y no con `if`? (2) Agreguen un quinto tipo de químico: ¿qué clases hay que tocar? (3) ¿Por qué `Contenedor` es abstracta? (4) ¿Qué pasa cuando sacan un químico del medio de la cola? (5) ¿Por qué solo comparan contra el último y no contra todos? (6) ¿Dónde está la lógica del juego y dónde la presentación?
2. Las tres del boceto: (7) ¿por qué `getImagen()` devuelve un `String` y no un `Color`? (8) ¿Quién decide si se puede canjear y cuánto suma? ¿Por qué no lo decide la ventana? (9) ¿Qué pasa si `puntajes.txt` no existe o no se puede escribir?
3. Sumar preguntas sobre la guía: dónde está la interfaz, el mapa, el `Comparable`, el `finally`, el hilo, el GRASP y el SOLID.
4. Sumar preguntas sobre la evolución: por qué se pasó a `ArrayList`, al mapa y a una excepción propia, con el antes y el después anotados.
5. En el ensayo, Sofia1 elige a quién le pregunta cada una; se contesta sin mirar.
6. Lo que alguien no puede contestar se repasa esa semana.

ESTÁ LISTA CUANDO

- Las cuatro contestan las nueve preguntas sin mirar.

☐ **Sofia2**

Repasar las dos secuencias para poder contarlas paso a paso en la defensa.

Con: draw.io

− **Cómo hacerlo**

PASOS

1. Cada una cuenta en voz alta una de las dos secuencias, siguiendo los números.

ESTÁ LISTA CUANDO

- Las cuatro pueden contar guardar y retirar sin mirar el diagrama.

- [ ] **Lyssie**

Prueba final de punta a punta: jugar desde el ejecutable entregado, en una computadora que no sea la de desarrollo.

Con: la computadora de otra integrante

Cómo hacerlo

PASOS

1. Descargar el código de `main` como si fuera el profe y correrlo desde cero.
2. Jugar una partida completa: dejar, retirar, canjear, perder y guardar las iniciales.
3. Anotar cualquier error y arreglarlo antes del viernes.

ESTÁ LISTA CUANDO

- El juego corre en una computadora limpia.

- [ ] **Todas**

Rotación: cada una explica en voz alta una clase que no escribió.

Cómo hacerlo

PASOS

1. Sortear las clases para que a nadie le toque la suya.
2. Explicar qué hace cada atributo y cada método, y por qué está en esa clase y no en otra.

ESTÁ LISTA CUANDO

- Las cuatro explicaron una clase ajena sin ayuda.

- [ ] **Todas**

Congelar el código el viernes 13/11. Desde ahí, solo arreglos de errores. Exportar el UML a PDF.

Con: draw.io

Cómo hacerlo

PASOS

1. **Viernes 13/11:** se mergea lo último aprobado y desde ahí solo entran arreglos de errores.
2. Exportar el diagrama unido y las secuencias a PDF (guía de draw.io).
3. Confirmar con el profe qué se entrega el 16/11 y en qué formato, y armarlo: UML, código, memoria.

TIENE QUE QUEDAR

- El código final, el UML en PDF, la memoria y lo que pida el profe.

ESTÁ LISTA CUANDO

- Se cumple el criterio de la entrega final.

CRITERIO DE LA ENTREGA FINAL (LUNES 16/11, GRUPO 1)

El juego corre, el UML coincide con el código, la memoria señala cada tema de la guía y las cuatro contestan las nueve preguntas sin mirar.

2 · PENDIENTES

Lo que falta decidir

Cada cosa se decide en la fecha indicada y se anota en «Ya decidido», acá abajo, así nadie programa con una idea distinta. Actualizado el 19/10 con lo que ya está en `main`.

QUÉ FALTA DECIDIR	PROPUESTA O DATO
ESTA SEMANA, CON EL UML	
Qué interfaz usan	Propuesta: `Dibujable` con `getImagen()`, para `Quimico` y `Visitante`.
Qué método se sobrecarga	Propuesta: `guardar(Quimico q)` en `Deposito`, al lado de `guardar(q, codigo)`: lo guarda en el primer contenedor donde entre. Tarea de Luz en el Sprint 4.
Agregar `getImagen()` a `Quimico`	Propuesta: sí. Hoy ningún químico lo tiene y la ventana del Sprint 5 lo necesita. Tarea del Sprint 4.
PREGUNTARLE AL PROFE (PENDIENTE DESDE EL 19/10)	
En qué grupo están: 1 (16/11) o 2 (30/11)	Mientras tanto, el plan apunta al 16/11.
Qué espera el profe de «destructores»	Java no tiene destructores.
Si cuenta el archivo, la interfaz gráfica o las dos	Unidad 6 de la guía: se acuerda con el docente.
Usar un archivo para el top 5	Archivos se ven el 2/11.
Qué se muestra en cada pre-entrega	Cada lunes es una pre-entrega sin nota.
DEL JUEGO	
Si guardar en un estante lleno es derrota o solo un aviso	Hoy el código lo trata como **derrota**: `guardar` devuelve `false` y `Juego` termina la partida. Confirmarlo a propósito o cambiarlo.
Cuántos visitantes hay y quién atiende	El boceto dice seis visitantes (las otras tres del equipo, la Dra. Mora, el Dr. Bigotes y el Ing. Paco) y una encargada que se elige al empezar. Hoy `Juego` tiene dos visitantes y ninguna encargada.
Nombres de los PNG	Propuesta: minúscula, sin tildes y con el tamaño al final (`mora_x4.png`, `mora_x2.png`), en `src/img/`. Están en el pull request #7. Faltan los íconos de los químicos y el fondo del mostrador.
MÁS ADELANTE	
Botón Volver al mostrador sin jugar	Sprint 4.
Qué más suma puntos, además del canje	Sprint 5, propuesta de Sofia2.

Qué hace el hilo	Antes del Sprint 6. Opciones: animar la llegada del visitante o la explosión, o guardar el top 5 en segundo plano. Consultarlo con el profe.

▼ Ya decidido

- **Cómo sabe cada químico con quién es compatible:** con `esCompatibleCon()` en cada tipo de químico, como en el UML que aprobó el profe el 5/10.
- **Qué rombo lleva cada relación:** composición entre `Deposito` y `Contenedor`; agregación entre `Contenedor` y `Quimico`.
- **Qué Java usan las cuatro: Eclipse Temurin 25**, descargado desde IntelliJ. El profe no pidió ninguna versión. Java 25 es la versión LTS (de soporte largo) más nueva y Temurin es gratis y recibe actualizaciones de seguridad. Todo lo que se ve en la cursada (herencia, `ArrayList`, excepciones, paquetes, Swing) funciona igual en esta versión. Anotarlo en el CLAUDE.md.
- **Cómo es un retiro:** Quien viene a retirar pide un tipo y una temperatura, y vale cualquier químico que coincida en las dos cosas. Si la jugadora saca otro químico, pierde. Anotarlo en la sección 2 del CLAUDE.md.
- **Dos pantallas de juego, mostrador y depósito:** Con el boceto del 5/10. La heladera tiene H1–H3 y al lado están los estantes E1–E3, de 4 lugares cada uno.
- **Canje:** Un estante lleno se puede canjear por puntos extra. Si después vuelven a buscar algo canjeado, se pierde.
- **Mejores puntajes:** Top 5 de esa computadora, con iniciales, puntaje y turno. Falta decidir cómo se guarda (fila de abajo).
- **Canje:** +40 fijos y no gasta el turno. Está en `Juego` como `PUNTOS_CANJE = 40` y `CANJE_GASTA_TURNO = false`; si el balanceo pide otro valor, se cambia ahí.
- **Retiros:** salen de los pendientes. Vuelve el mismo visitante que dejó el químico y pide ese tipo y esa temperatura.
- **«No lo tenías»:** si llega un retiro y en el depósito no queda ninguno de ese tipo y esa temperatura, se pierde en ese momento. Lo hace `Juego.revisarQueLoTenes()`.
- **Fin de la partida:** se juega hasta perder.
- **Pedidos al azar:** con `Math.random()` y casteo a `int`, en `Juego`.
- **Quién avanza el turno:** `Juego`, solo. La ventana (o la consola) llama a `juego.jugar(codigo, posicion)`; si da `false`, muestra `juego.getMotivo()`. `siguienteTurno()` es privado y nadie de afuera lo llama. Resuelve la duda 2 del Sprint 2.
- **Dónde se anotan las decisiones:** en esta lista. El CLAUDE.md que nombran las tareas viejas no se sube al repositorio.
- **Fechas de entrega:** Cada lunes es una pre-entrega sin nota, salvo el 12/10 y el 9/11 (feriados). La final y la defensa son el día del 2.° parcial: 16/11 (Grupo 1) o 30/11 (Grupo 2).

3 · PARA CONSULTAR

Reglas del juego

Lo que dice la sección 2 del CLAUDE.md, más lo que cambió con el boceto del 5/10. Si algo de acá cambia, se cambia primero en el CLAUDE.md.

Un turno: mostrador → depósito → mostrador

Químicos y depósito

1. **Mostrador:** llega un visitante. Deja un químico (tipo y si necesita frío o calor) o vuelve a buscar uno (tipo y temperatura).
2. La jugadora toca **Ir al depósito**.
3. **Depósito:** para dejar, hace clic en el estante donde va; para retirar, hace clic en el químico que entrega.
4. El modelo valida. Si está bien, pasa el turno y vuelve al mostrador con el próximo pedido. Si no, va a la pantalla final con el motivo.

Cuatro tipos, ni uno más: `Acido`, `Base`, `Inflamable` y `Oxidante`.

Frío: una heladera con tres estantes, H1, H2 y H3. **Calor:** al lado, los estantes E1, E2 y E3. Cada uno tiene 4 lugares.

En el código siguen siendo 6 `Contenedor`: 3 `Heladera` y 3 `Estante`. En pantalla los de la heladera también se ven como estantes.

Guardar en el lugar de la temperatura equivocada es jugada inválida.

Compatibilidad: se compara solo contra el último

Antes de guardar, el químico nuevo se compara contra **el último guardado en ese estante** (el de más atrás). Si cada guardado anterior se validó, el resto de la fila ya está bien.

ÚLTIMO \ NUEVO	ACIDO	BASE	INFLAMABLE	OXIDANTE
Acido	Se pudren	Explota	Va	Va
Base	Explota	Se pudren	Va	Va
Inflamable	Va	Va	Se pudren	Explota
Oxidante	Va	Va	Explota	Se pudren

Truco para acordarse: hay dos grupos, {Inflamable, Oxidante} y {Acido, Base}. Solo se puede alternar entre grupos: nunca dos del mismo grupo pegados, nunca dos iguales.

La cola de cada estante

El químico nuevo **siempre entra al final** (última posición ocupada + 1). Nunca en un hueco del medio.

Cuando se saca uno del medio, **los de atrás se corren hacia adelante** para tapar el hueco.

No es un bug: al correrse pueden quedar pegados dos incompatibles, y explota. Retirar es una jugada riesgosa.

Retiros DECIDIDO

El pedido es un **tipo** y una **temperatura**, y vale cualquier químico que coincida en las dos cosas.

Si saca otro químico, pierde. Si lo que queda pegado al correrse la cola es incompatible, también pierde.

Decidido y programado en `Juego`: quien retira es un visitante que antes dejó ese químico, y el retiro sale de la lista de **pendientes**. Si no queda ninguno de ese tipo y esa temperatura en el depósito, se pierde en ese momento.

Canje DECIDIDO

Cuando un estante está lleno aparece el botón **Canjear**. Es opcional.

Canjear vacía ese estante y suma **+40** fijos. No gasta el turno.

Mejores puntajes DECIDIDO

Los 5 mejores de esa computadora: iniciales de 3 letras, puntaje y turno. Si la partida entra en el top 5, la pantalla final pide las iniciales.

Propuesto: se guardan en `puntajes.txt`, al lado del juego, una línea por puesto (`LUZ;340;31`). Si

Lo canjeado ya no se puede entregar: si después vuelven a buscar algo que estaba ahí, se pierde. No es un bug.

el archivo no existe o falla, la tabla arranca vacía y se sigue jugando. Consultarlo con el profe antes de programarlo.

Perder siempre explica por qué

Los motivos los arma el modelo y nombran el estante con su código, como se ve en pantalla (H2, E1).

FORMA	TÍTULO	MOTIVO DE EJEMPLO
Dos iguales	**¡SE PUDRIERON!**	Pusiste un oxidante detrás de otro oxidante en E1.
Explosión	**¡EXPLOTÓ!**	Pusiste una base detrás de un ácido en H2.
Temperatura	**¡SE ARRUINÓ!**	Guardaste en la heladera un inflamable que necesita calor.
Retiro equivocado	**¡ESE NO ERA!**	Te pidieron un inflamable en calor y entregaste una base de E3.
Al correrse	**¡EXPLOTÓ!**	Al sacar el inflamable de H1, el ácido y la base quedaron pegados.
No lo tenías	**¡NO LO TENÍAS!**	El Dr. Bigotes volvió por su oxidante, pero canjeaste el estante E1.
Estante lleno	Pendiente	Hoy el código lo trata como derrota. Falta confirmarlo (ver «Lo que falta decidir»).

El boceto da por hecho que se juega hasta perder. Lo que falta decidir está en Lo que falta decidir.

3 · PARA CONSULTAR

Lo que pide la guía del profe

Cada tema tiene que poder señalarse en el código. La tabla dice dónde va cada uno en Alquimia y es la base de la memoria técnica.

Cuándo se ve cada tema en clase

`final` y clases internas: 5/10. `ArrayList` y `HashMap`: 19/10. Excepciones: 26/10. Archivos y paquetes: 2/11. Swing e hilos: 30/11, después de la entrega del Grupo 1, así que se aprenden por cuenta propia.

Qué se entrega y cómo se aprueba

Diagramas UML (clases y secuencia), código fuente completo que compile y corra, una **memoria técnica** breve (el diseño y dónde se aplica cada tema) y la defensa oral.

Con **8 o más** en la entrega final, aprobación directa de la cursada; entre 4 y 8, se aprueba.

TEMA DE LA GUÍA	DÓNDE VA EN ALQUIMIA	SPRINT	ESTADO
UNIDAD 1 · POO Y UML			
Diagrama de clases del sistema completo	El diagrama unido, con las clases del boceto y las que suma esta tabla.	S2 y S6	EN EL DISEÑO

Al menos un diagrama de secuencia	Guardar y retirar.	S1 y S6	EN EL DISEÑO
Relaciones con cardinalidad, direccionalidad y composición	`Deposito` ◆ `Contenedor` (1 a 6), agregación `Contenedor` – `Quimico` (1 a 0..4) y flechas navegables.	S1	EN EL DISEÑO
Constructores y destructores	Constructores en todas las clases. Java no tiene destructores: preguntarle al profe qué espera ver.	S2	A CONSULTAR
Tipos primitivos y wrapper	`int` y `boolean` en todo el modelo; `Integer.parseInt(...)` al leer `puntajes.txt` y wrappers dentro de las colecciones.	S4 a S6	PROPUESTO
UNIDAD 2 · EL LENGUAJE			
Estructura, sintaxis y buenas prácticas	Convenciones de la sección 6 del CLAUDE.md y revisión cruzada.	Siempre	EN EL DISEÑO
Primitivos y de referencia, casteo	Casteo al elegir al azar: `(int) (Math.random() * 4)` en `Juego.generarPedidoDeDejar()`.	S3	EN EL CÓDIGO
Arrays	`Quimico[4]` en `Contenedor`, `Contenedor[6]` en `Deposito` y los pendientes de `Juego`.	S2 y S3	EN EL CÓDIGO
UNIDAD 3 · CLASES Y OBJETOS			
Herencia	`Quimico` y sus 4 tipos; `Contenedor`, `Heladera` y `Estante`; los paneles extienden `JPanel` (Sprint 4).	S2	EN EL CÓDIGO
Sobrecarga y sobreescritura; sobrecarga de constructores	Sobreescritura: `esCompatibleCon` y `aceptaTemperatura` (en el código). Sobrecarga de constructores: `Contenedor(codigo)` y `Contenedor(codigo, lugares)` (falta). Sobrecarga de un método: `guardar(Quimico q)` en `Deposito` (falta).	S2 y S4	EN PARTE
`public`, `private`, `protected` y default	Los tres primeros ya están en el código. Default (sin modificador): `vaciar()` de `Contenedor`, que solo usa el depósito. Hoy es `public`.	S2 y S4	EN PARTE
Una clase abstracta y una interfaz, con su contrato	Abstractas: `Quimico` y `Contenedor`. **Falta la interfaz.** Propuesta: `Dibujable`, con `getImagen()`, que implementan `Quimico` y `Visitante`.	S2	FALTA
Polimorfismo	`esCompatibleCon` y `aceptaTemperatura`.	S2	EN EL CÓDIGO
`super()` y `this()`	`super(necesitaFrio)` en cada químico y `super(codigo)` en `Heladera` y `Estante` (en el código); `this(codigo, LUGARES)` uniendo los dos constructores de `Contenedor` (falta).	S2 y S4	EN PARTE
`static`	Constantes `static final`: `LUGARES` en `Contenedor` y `PUNTOS_CANJE` en `Juego` (en el código); tamaño del top y la clase `Colores` (Sprint 5).	S2 a S5	EN PARTE

Un patrón GRASP y un principio SOLID, nombrados en la documentación	GRASP: Experto (cada químico decide su compatibilidad), Controlador (`Juego`) o Creador (`Deposito` crea sus contenedores). SOLID: abierto/cerrado (el quinto tipo) o responsabilidad única (solo `TablaPuntajes` toca archivos).	S6 y S7	PROPUESTO
Opcional y valorado: inmutables, enum, clases internas	`Visitante` inmutable (atributos `final` , sin setters) y el enum `TipoPedido` {DEJAR, RETIRAR}, los dos en el código. Los listeners como clases internas llegan con la ventana (Sprint 4).	S3 y S4	EN PARTE
UNIDAD 4 · COLECCIONES			
Colecciones genéricas: listas, mapas	`ArrayList<Quimico>` en `Contenedor` y en los pendientes. Mapa: `HashMap<String, Contenedor>` en `Deposito` , para buscar por código.	S4	PROPUESTO
`Comparable` y/o `Comparator`	Una clase `Puntaje` (iniciales, puntos y turno) que implementa `Comparable<Puntaje>` para ordenar el top 5.	S5	FALTA
Recorrido con for-each	Pendientes, paneles y puntajes.	S4	PLANEADO
UNIDAD 5 · EXCEPCIONES			
`try` , `catch` y `finally`	La ventana y la partida por consola atrapan la excepción propia; `finally` cierra el archivo de puntajes.	S5 y S6	PLANEADO
`throw` y `throws`	El modelo lanza `JugadaInvalidaException` y lo declara con `throws` .	S5	PLANEADO
Una excepción propia y las comunes	`JugadaInvalidaException` ; `NumberFormatException` si una línea de `puntajes.txt` está rota y `FileNotFoundException` si no existe.	S5 y S6	PLANEADO
UNIDAD 6 · ENTRADA/SALIDA, SWING, ARCHIVOS E HILOS			
Un archivo o, si no, interfaz gráfica con eventos (se acuerda con el docente)	Hay las dos: la ventana Swing y `puntajes.txt` . Acordar con el profe cuál cuenta.	S4 a S6	A CONSULTAR
Hilos de ejecución separados	**No está en el diseño.** Opciones que no cambian las reglas: animar en un hilo aparte la llegada del visitante o la explosión, o guardar el top 5 en un hilo aparte.	S6	FALTA
ENTREGABLES			
Memoria técnica	Documento breve: el diseño y esta misma tabla, con la clase y el método donde está cada tema.	S6 y S7	FALTA

4 · CÓMO TRABAJAMOS

Encuentros y revisión

Tres encuentros por semana. Lo que no puede faltar es la revisión del domingo.

Lunes

Después de clase · 30 minutos

Repasar si se cumplió el criterio, repartir las tareas y actualizar la sección 5 del CLAUDE.md con lo visto en clase.

Jueves

Por mensaje o llamada · 15 minutos

Qué terminé, qué sigo y dónde estoy trabada. Quien lleve más de un día trabada pide ayuda, no espera al domingo.

Domingo

Por Meet · 1 hora

Demo y revisión cruzada. Cada una explica en voz alta el código de otra; lo que no se puede explicar se reescribe más simple.

Quién revisa a quién

Cambia cada sprint, así al final todas leyeron el código de todas. Git: una rama por tarea, `main` siempre compila y nada entra sin que otra lo haya leído.

REVISORA	S1	S2	S3	S4
Sofia1	Lyssie	Luz	Sofia2	Lyssie
Sofia2	Sofia1	Lyssie	Luz	Sofia1
Lyssie	Luz	Sofia2	Sofia1	Luz
Luz	Sofia2	Sofia1	Lyssie	Sofia2

Qué mirar en cada revisión

1. ¿Hay algún `if` que pregunte el tipo de químico fuera de las clases de químico?
2. ¿Algún listener de la ventana decide una regla?
3. ¿Algún `for` recorre el array hasta `.length` en vez de hasta `cantidad`?
4. ¿Se usa algo que todavía no se vio en clase?
5. ¿La ventana dibuja con `getImagen()` sin ningún `if` por tipo?
6. ¿El botón Canjear solo se muestra, sin decidir nada?
7. ¿Algo fuera de `TablaPuntajes` toca archivos?

4 · CÓMO TRABAJAMOS

Herramientas y guías

Todo gratis y en Java. Las tareas mandan a estas guías cuando hace falta; los nombres de los botones pueden cambiar un poco según la versión.

PROGRAMAR EN JAVA

IntelliJ IDEA, versión gratuita

Desde la 2025.3 es un solo producto y todo lo de Java es gratis, sin activar nada (JetBrains). Si el profe usa otro IDE en clase, usen ese.

INSTALAR JAVA

El mismo JDK para las cuatro

Con versiones distintas aparecen errores que parecen del código y no lo son.

LA VENTANA

Swing escrito a mano

Nada de editores visuales de ventanas: generan código que después no pueden explicar en la defensa.

UML

draw.io

Gratis, sin cuenta y con figuras de UML 2.5 (draw.io). Guardando el archivo en Google Drive lo editan entre todas.

COMPARTIR EL CÓDIGO

GitHub, desde IntelliJ o GitHub Desktop

Git con botones en vez de comandos: ramas, commits y pull requests para la revisión cruzada. Las cuatro trabajan en el

CÓMO SE VE EL JUEGO

El boceto de pantallas

Las cuatro pantallas, los personajes, la paleta y la guía para armarlo en Swing. Es privado: Luz lo comparte desde Share.

repositorio del grupo, nunca en un fork.

DIBUJOS EN PNG

Un editor de pixel art gratuito

Por ejemplo Piskel, que corre en el navegador. Se dibuja chico y se exporta ya agrandado, porque agrandarlo desde Java lo deja borroso.

TIPOGRAFÍAS

Google Fonts

Pixelify Sans y Nunito, gratis (licencia OFL). Se bajan los TTF y van en `src/fuentes/`.

COORDINARSE

WhatsApp y Meet

WhatsApp para el jueves, Meet con pantalla compartida para el domingo.

▼ Git con GitHub Desktop (o desde IntelliJ)

> **Desde IntelliJ:** `Ctrl+T` trae lo de GitHub (hacerlo siempre antes de empezar y después de tocar algo en la web), `Ctrl+K` hace el commit y `Ctrl+Shift+K` el push. La rama se cambia con el nombre que aparece arriba a la izquierda (o abajo a la derecha). Si GitHub no te deja subir una rama, no hagas un fork: pedile a Luz que te agregue como colaboradora.

1. **Crear el proyecto (lo hace una sola).** En IntelliJ, crear el proyecto Java (guía de IntelliJ, paso 1).
2. **Convertirlo en repositorio y publicarlo (la misma).** En GitHub Desktop: File › Add local repository, elegir la carpeta del proyecto y aceptar crear un repositorio ahí. Crear un archivo `.gitignore` con estas líneas, para no subir archivos de cada computadora: `out/` y `.idea/workspace.xml`. Hacer el primer commit y tocar **Publish repository**.
3. **Invitar y clonar.** En github.com, en el repositorio: Settings › Collaborators › Add people, y agregar a las otras tres por su usuario. Cada una acepta el mail de invitación y en GitHub Desktop hace File › Clone repository, pestaña GitHub.com, elige el repositorio y una carpeta. Después lo abre en IntelliJ con File › Open.
4. **Una rama por tarea.** Antes de empezar, en GitHub Desktop: Current Branch › New Branch, con un nombre corto que diga la tarea (por ejemplo, `contenedor`). Siempre crearla desde `main` actualizado.
5. **Commit.** Cuando algo compila y funciona, en GitHub Desktop aparecen los archivos cambiados a la izquierda. Escribir abajo un resumen corto ("Contenedor: guardar y sacar") y tocar **Commit to** la rama. Después, **Publish branch** la primera vez y **Push origin** las siguientes.
6. **Pull request.** Tocar **Preview Pull Request** y después **Create pull request**: se abre GitHub. Escribir qué hace el cambio y cómo probarlo, y en Reviewers elegir a la revisora de la tabla.
7. **Revisar (la revisora).** En el pull request, pestaña Files changed: leer cada archivo con la lista "Qué mirar en cada revisión". Comentar en la línea lo que haya que cambiar. Cuando está bien: Review changes › Approve.
8. **Mergear.** Con la aprobación, la dueña toca **Merge pull request** en GitHub. Nada entra a `main` sin aprobación: GitHub Desktop no lo impide, es un acuerdo del grupo.
9. **Traer lo de las demás.** Para actualizar `main`: pasar a `main` en Current Branch, tocar **Fetch origin** y después **Pull origin**. Para llevar esos cambios a una rama propia: pasar a la rama y, en Current Branch, **Choose a branch to merge into** la rama › `main`.

▼ draw.io en Google Drive

1. **Crear el archivo (una sola).** Entrar a app.diagrams.net, elegir guardar en Google Drive, autorizar, crear un diagrama nuevo en blanco y ponerle nombre.
2. **Compartirlo.** En Google Drive, clic derecho sobre el archivo › Compartir, y agregar a las otras tres como editoras. Para abrirlo: clic derecho › Abrir con › diagrams.net.
3. **Páginas.** Abajo, el **+** agrega una página. Cada una trabaja en la suya para no pisarse.
4. **Figuras de UML.** En el panel izquierdo, la sección UML. Si no aparece, agregarla desde "Más figuras". La figura de clase tiene tres partes: nombre, atributos y métodos.
5. **Clase o método abstracto.** El nombre en cursiva.
6. **Herencia.** Línea continua con punta de triángulo vacío, desde la subclase hacia la superclase.
7. **Multiplicidad.** Doble clic sobre la línea, cerca de cada punta, para escribir el número (1, 6, 0..4).
8. **Exportar.** Archivo › Exportar como › PDF (o PNG).

▼ IntelliJ IDEA

1. **Proyecto nuevo.** File › New › Project › Java. Build system: IntelliJ. JDK: Eclipse Temurin 25 (si no aparece en la lista, Download JDK › Version 25 › Vendor Eclipse Temurin). Desmarcar el código de ejemplo si no hace falta.
2. **Abrir el proyecto del grupo.** File › Open y elegir la carpeta que clonó GitHub Desktop.
3. **Revisar el JDK.** File › Project Structure › Project: tiene que decir Eclipse Temurin 25 (versión 25).
4. **Clase nueva.** Clic derecho sobre `src` › New › Java Class, y escribir el nombre sin tildes (sección 6). Un archivo por clase.
5. **Correr un main.** El triángulo verde al lado de `public static void main`. Lo que imprime aparece abajo, en la consola.
6. **Errores.** Lo subrayado en rojo no compila: pasar el mouse por encima para leer el error antes de pedir ayuda.

4 · CÓMO TRABAJAMOS

Si algo se complica

El riesgo más caro es empezar la ventana con el modelo a medias: cada error de reglas se vuelve el doble de difícil de encontrar.

SI PASA ESTO	QUÉ HACER
El 5/10 el profe pide cambios grandes al UML	Se corrigen lunes y martes; el código arranca el miércoles con el diagrama corregido.
El 11/10 las reglas todavía no andan por consola	No se empieza el Sprint 3. Todas ayudan a terminar el modelo y el Sprint 6 absorbe el atraso.
Herencia se sigue postergando en clase	No se espera: el diagrama del Sprint 1 depende de ella. Se estudia por cuenta propia, igual que Swing.
Swing cuesta más de lo previsto	Alcanza con un botón por casilla y texto. Lo visual del Sprint 5 es lo primero que se recorta.

`ArrayList` o la excepción rompen algo	Se descarta esa rama y se sigue con la versión anterior. En la defensa se cuenta qué se intentó.
Se pierde siempre antes del turno 10	Aplicar la sección 8 en orden: 3 lugares, permitir rechazar, más retiros. No inventar reglas nuevas.
Una integrante se atrasa o se enferma	Toma la tarea quien la revisa ese sprint, que ya conoce el código.
Alguien no puede explicar una parte	Se reescribe más simple esa misma semana, no la anterior a la entrega.
Los PNG o la escena llevan más tiempo de lo previsto	El mostrador queda con rectángulos de color y texto. Los dibujos son lo primero que se recorta.
El profe no acepta archivos	El top 5 dura solo mientras el juego está abierto.
`CardLayout` se complica	Mostrador y depósito van en una sola pantalla, como el primer boceto.
El canje desbalancea el juego	Se ajusta su valor; no se agregan reglas.